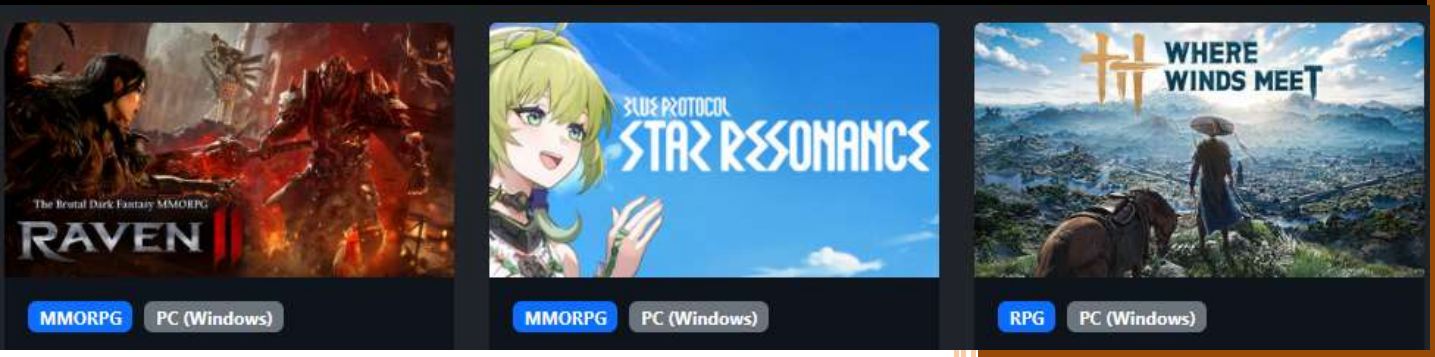
F2P CATALOG

Catálogo web de juegos Free-to-play



Autores: Pablo Laya Bolívar, Pablo Pérez Lagares

CFSG – Desarrollo de Aplicaciones Web

Tutor: Alejandro Bastida Castillo

Convocatoria ordinaria - Curso 2025/2026

Mayo de 2026

Índice

Resumen ejecutivo 4
1. Introducción 5
2. Objetivos y alcance 7
2.1 Objetivo general 7
2.2 Objetivos específicos 7
2.3 Alcance y fuera de alcance 8
3. Justificación y marco teórico 9
3.1 Justificación del proyecto 9
3.2 Aplicaciones web renderizadas en servidor 9
3.3 Flask y patrón MVC 9
3.4 Base de datos relacional y ORM 10
3.5 Frontend con Jinja2, Bootstrap y CSS propio 10
3.6 Seguridad básica en aplicaciones web 11
3.7 Contenedores y despliegue como parte del marco técnico 11
4. Análisis y planificación del proyecto 12
4.1 Usuarios y roles 12
4.2 Requisitos funcionales 12
4.3 Requisitos no funcionales 13
4.4 Historias de usuario 14
4.5 Casos de uso 15
4.6 Planificación 16
4.7 Riesgos 17
4.8 Priorización de funcionalidades 18
4.9 Organización del trabajo 18
5. Diseño del sistema 20
5.1 Arquitectura general 20
5.2 Modelo de datos 21
5.3 Diccionario de datos 21
5.4 Diseño de rutas y endpoints 22
5.5 Uso de la API FreeToGame 23
5.6 Diseño de interfaz y experiencia de usuario 24

5.7 Contratos entre backend y frontend 28
5.8 Estados vacíos y errores en la interfaz 28
6. Desarrollo e implementación 29
6.1 Entorno de desarrollo 29
6.2 Estructura del repositorio 29
6.3 Factoría de aplicación y extensiones 30
6.4 Modelos y validaciones 30
6.5 Catálogo, filtros y paginación 30
6.6 Autenticación y sesiones 31
6.7 Biblioteca y reseñas 31
6.8 Seed y sincronización con FreeToGame 32
6.9 Frontend y plantillas 32
6.10 Decisiones técnicas destacadas 32
6.11 Fechas, zona horaria y presentación de datos 33
6.12 Manejo de errores 33
7. Pruebas y validación 34
7.1 Enfoque de pruebas 34
7.2 Casos de prueba 34
7.3 Resultados y observaciones 35
7.4 Pruebas por rol 37
7.5 Evidencias aportadas para la defensa 37
8. Despliegue y puesta en producción 38
8.1 Despliegue local con Docker Compose 38
8.2 Despliegue de producción 38
8.3 Variables de entorno 39
8.4 Puesta en marcha en producción 40
8.5 Copias de seguridad y mantenimiento 40
8.6 Checklist de despliegue 40
9. Seguridad y protección de datos 42
9.1 Medidas técnicas aplicadas 42
9.2 Validación y control de datos 42
9.3 Protección de datos y RGPD básico 42

9.4 Limitaciones de seguridad 43
9.5 Buenas prácticas pendientes 43
10. Manual de usuario 44
10.1 Acceso a la aplicación 44
10.2 Consultar catálogo 45
10.3 Ver ficha de juego 45
10.4 Registro e inicio de sesión 45
10.5 Gestionar biblioteca personal 46
10.6 Publicar y editar reseñas 46
10.7 Perfil de usuario 47
10.8 Panel de administración 47
10.9 Recorrido recomendado para una demo 48
11. Manual técnico 49
11.1 Requisitos previos 49
11.2 Instalación local con Docker 49
11.3 Carga de datos inicial 49
11.4 Ejecución fuera de Docker 49
11.5 Archivos de configuración 50
11.6 Mantenimiento técnico 50
11.7 Solución de problemas 50
11.8 Procedimiento seguro de actualización 51
11.9 Estructura de plantillas y parciales 51
12. Conclusiones y líneas futuras 53
12.1 Conclusiones 53
12.2 Líneas futuras 53
12.3 Relación con módulos de DAW 54
Bibliografía y webgrafía 55
Anexos 55
Anexo I. Resumen de comandos principales 56
Anexo II. Resumen de entidades 56
Anexo III. Glosario breve 57

Resumen ejecutivo

F2P Catalog es una aplicación web para consultar, valorar y organizar juegos free-to-play. La necesidad que se aborda es sencilla: la información de estos juegos aparece repartida entre tiendas, páginas oficiales, listados externos y comunidades, y no siempre resulta cómodo comparar títulos o guardar los que interesan. La solución centraliza un catálogo obtenido desde FreeToGame, lo guarda en PostgreSQL y lo muestra desde una interfaz renderizada en servidor. La aplicación permite buscar y filtrar juegos, abrir una ficha pública, crear cuenta, iniciar sesión, gestionar una biblioteca personal, publicar una reseña propia por juego y acceder a un panel básico de administración. Se ha desarrollado con Python 3.11, Flask, Jinja2, Bootstrap, SQLAlchemy, Flask-Login, Flask-Bcrypt, Flask-WTF y PostgreSQL. El despliegue se ha preparado con Docker Compose, Gunicorn y la URL pública https://f2p.prodelaya.dev/. La versión entregada permite consultar el catálogo público y validar los flujos principales; sus límites principales son la ausencia de tests automatizados, migraciones de base de datos y política de privacidad visible.

Bloque	Resumen del resultado
Producto	Aplicación web SSR para catálogo, biblioteca personal, reseñas y administración.
Usuarios	Visitante anónimo, usuario registrado y administrador.
Backend	Flask, SQLAlchemy, Flask-Login, Flask-Bcrypt, Flask-WTF y servicio FreeToGame.
Frontend	Plantillas Jinja2, Bootstrap 5, CSS propio y JavaScript vanilla puntual.
Base de datos	PostgreSQL con tablas users, games, reviews y user_library.
Estado	versión operativa y desplegable y documentado, con pruebas manuales y mejoras pendientes concretas.

1. Introducción

El mercado del videojuego se mantiene como un sector amplio, con catálogos cada vez más repartidos entre tiendas, webs oficiales, comunidades y servicios especializados. Dentro de ese contexto, los juegos free-to-play permiten empezar a jugar sin pagar una licencia inicial y suelen incorporar compras opcionales, pases de temporada o contenido cosmético. Informes sectoriales como el Global Games Market Report de Newzoo describen el videojuego como un mercado segmentado por plataforma y con un volumen suficiente para justificar herramientas de consulta y organización (Newzoo, 2024).

Una parte de los juegos aparece en tiendas como Steam o Epic Games, otra parte se encuentra en páginas oficiales y otra parte se anuncia en comunidades, blogs o vídeos. Además, algunos juegos están orientados a PC, otros funcionan en navegador y otros mezclan varias plataformas. La API de FreeToGame se eligió porque ofrece un catálogo abierto de juegos gratuitos y proporciona información estructurada en formato JSON, como título, género, plataforma, descripción, imágenes y requisitos. La consulta de la documentación y de la API se realizó durante mayo de 2026, por lo que las cifras concretas del catálogo pueden variar con el tiempo (FreeToGame, s. f.).

El problema no es que no exista información, sino que no siempre está ordenada desde el punto de vista del usuario que solo quiere descubrir un juego, ver si le interesa, guardarlo y leer opiniones. Si cada consulta obliga a saltar entre varias webs, se pierde tiempo y es más difícil comparar juegos por género, plataforma o valoración. En un proyecto de DAW, este contexto encaja bien porque permite trabajar con una aplicación web completa: backend, frontend, base de datos, seguridad básica y despliegue. Un ejemplo de uso sería un visitante que entra buscando juegos de navegador, filtra por plataforma, abre la ficha de un título, lee los requisitos y decide crear una cuenta para guardarlo en su biblioteca. Ese recorrido concreto fue el que se tomó como referencia al revisar la home, el catálogo y la ficha pública.

F2P Catalog nace para cubrir ese escenario de forma sencilla. No pretende competir con una tienda real ni con una plataforma profesional de videojuegos. La idea es construir una aplicación web académica, pero funcional, que centralice el catálogo en una base de datos local, permita consultar fichas y añada una capa de usuarios, biblioteca personal y reseñas. De esta forma, el catálogo no se queda solo en una lista estática de juegos, sino que incorpora interacción y pequeñas funcionalidades de comunidad.

También se ha decidido que la aplicación sea server-rendered con Flask y Jinja2. Esto significa que el servidor genera las páginas HTML y las entrega al navegador, en lugar de crear una SPA separada. Esta decisión reduce complejidad para un proyecto de 2.º de DAW y permite centrarse en aprender bien rutas, formularios, sesiones, seguridad CSRF, consultas a base de datos y plantillas reutilizables.

A partir de ese contexto, el proyecto se plantea como una versión inicial desplegada: un alcance reducido, pero con suficientes piezas reales para demostrar análisis, diseño, implementación, pruebas, seguridad y despliegue sin intentar copiar todas las funciones de una plataforma profesional.

Antes del proyecto	Después con F2P Catalog
Información repartida entre distintas webs y fuentes.	Catálogo centralizado con datos cacheados en PostgreSQL.
El usuario consulta juegos sin guardar preferencias.	El usuario puede crear biblioteca personal y marcar estados.
Las opiniones dependen de webs externas.	Los usuarios pueden dejar una reseña por juego y editarla.
La actualización de datos no está bajo control del proyecto.	El administrador puede lanzar una actualización manual del catálogo.

Tabla 1. Comparación simple entre el escenario inicial y la solución desarrollada.

2. Objetivos y alcance

Después de explicar el problema, se concretan los objetivos del trabajo. Se intenta que sean verificables y que no prometan más de lo que realmente se ha construido. Esta parte es importante porque marca el límite de la versión entregada y evita mezclar funcionalidades reales con ideas futuras.

2.1 Objetivo general

Desarrollar una aplicación web server-rendered para descubrir, consultar, valorar y organizar juegos free-to-play, integrando datos de una API externa, persistencia en base de datos, autenticación de usuarios, funcionalidades privadas y un panel básico de administración.

2.2 Objetivos específicos

- OE1. Diseñar un modelo de datos sencillo y coherente para usuarios, juegos, reseñas y biblioteca personal.
- OE2. Consumir la API externa FreeToGame y cachear los juegos en una base de datos local para no depender de llamadas externas en cada visita.
- OE3. Implementar rutas públicas para inicio, catálogo, filtros, búsqueda, paginación y ficha detallada de juego.
- OE4. Implementar registro, inicio de sesión y cierre de sesión mediante Flask-Login y contraseñas hasheadas con bcrypt.
- OE5. Permitir que el usuario registrado gestione su biblioteca personal con estados como quiero jugar, jugando y jugado.
- OE6. Permitir que el usuario cree, edite y elimine una reseña propia por juego, con valoración de 1 a 5.
- OE7. Crear un panel de administración para moderar reseñas y actualizar manualmente el catálogo.
- OE8. Diseñar un frontend responsive con Jinja2, Bootstrap y CSS propio sin romper los contratos del backend.
- OE9. Aplicar medidas de seguridad básicas: CSRF, validaciones en servidor, control de roles y redirecciones seguras.
- OE10. Documentar instalación, despliegue, pruebas, decisiones técnicas y posibles mejoras futuras.

Objetivo	Cómo se verifica
OE1-OE2	Modelo de datos real, seed de FreeToGame y catálogo cacheado en PostgreSQL.
OE3	Rutas públicas /, /catalogo y /juego/<id> con filtros y paginación.
OE4-OE7	Flujos de registro, login, biblioteca, reseñas y administración probados manualmente.
OE8	Plantillas Jinja2, Bootstrap, CSS propio y revisión

	responsive.
OE9	CSRF, bcrypt, login_required, admin_required y validaciones server-side.
OE10	Manual técnico, comandos Docker, despliegue y bibliografía revisada.

La tabla siguiente relaciona cada objetivo con la evidencia usada para comprobarlo durante la defensa y en la propia memoria.

2.3 Alcance y fuera de alcance

Dentro del alcance	Fuera del alcance y motivo
Catálogo con búsqueda, filtros y paginación.	Pagos o compras: no aporta al objetivo DAW y exigiría pasarela real.
Ficha detallada con descripción, requisitos, enlaces y capturas.	Integración con Steam/Epic: depende de APIs externas y permisos ajenos.
Registro, login, logout y sesiones de usuario.	Aplicación móvil nativa: queda fuera del ciclo web y duplicaría esfuerzo.
Biblioteca personal con tres estados controlados.	Listas personalizadas avanzadas: aumentan el alcance sin ser necesarias para el alcance de clase.
Reseñas con rating y texto validado.	Moderación automática: requiere reglas más complejas y no era prioritaria.
Panel admin para borrar reseñas y actualizar catálogo.	Backoffice completo: estadísticas y auditoría quedan como mejora futura.
Despliegue con Docker Compose y Gunicorn.	Alta disponibilidad o balanceo: excesivo para una demo académica.
Pruebas manuales documentadas.	Suite E2E completa: se reconoce como mejora futura por tiempo y alcance.

El alcance se ha limitado de forma consciente. En un proyecto de clase no tiene sentido intentar implementar todas las funciones de una plataforma profesional. Se ha priorizado que las funciones incluidas estén conectadas entre sí y sean demostrables durante la defensa.

3. Justificación y marco teórico

3.1 Justificación del proyecto

El proyecto se justifica por dos motivos principales. El primero es funcional: FreeToGame ofrece datos estructurados de juegos gratuitos, pero el usuario sigue necesitando una interfaz propia para buscar, filtrar, guardar juegos y añadir opiniones. El segundo es académico: la aplicación permite aplicar varias competencias del ciclo DAW en un único producto, desde el diseño de base de datos hasta el despliegue con contenedores.

El caso de uso elegido es suficientemente cercano para que se pueda entender sin conocimientos de empresa. Un usuario entra, busca un juego, filtra por género o plataforma, lee la ficha, lo añade a su biblioteca y puede dejar una reseña. Por detrás, el sistema trabaja con sesiones, roles, formularios protegidos, validaciones, consultas a base de datos y un servicio que consume una API externa.

La decisión de cachear los juegos en PostgreSQL tiene bastante peso en el proyecto. Si la aplicación consultara FreeToGame en cada petición del catálogo, dependería demasiado de la disponibilidad y velocidad de una API externa. Guardar los datos localmente permite paginar, filtrar, ordenar y mostrar fichas desde la base de datos propia. Además, permite que el administrador actualice el catálogo cuando haga falta.

Como evidencia técnica, el catálogo no se muestra directamente desde la API externa en cada visita: primero se ejecuta el seed de juegos, se guardan los datos en PostgreSQL y después las rutas públicas consultan la base de datos local. Esto se puede comprobar en seeds/seed_games.py y en las rutas de app/routes/games.py.

3.2 Aplicaciones web renderizadas en servidor

Una aplicación renderizada en servidor genera el HTML desde el backend y lo devuelve al navegador ya preparado. En este proyecto, Flask recibe la petición, consulta los datos necesarios, renderiza una plantilla Jinja2 y envía la página completa. Este enfoque es más clásico que una SPA, pero sigue siendo válido para muchas aplicaciones y resulta bastante claro para aprender.

La ventaja principal para este proyecto es que no se necesita construir un frontend separado con React o Vue. También se simplifica la autenticación porque las sesiones de Flask-Login se integran directamente con las rutas y las plantillas. La desventaja es que algunas interacciones dinámicas no son tan fluidas como en una SPA, aunque para esta versión inicial no era un problema importante.

3.3 Flask y patrón MVC

Flask es un microframework de Python que permite crear aplicaciones web con bastante libertad (Pallets, s. f.). En el proyecto se ha usado con el patrón Application Factory, de forma que la función create_app() carga configuración, inicializa extensiones, registra rutas y configura manejadores de error.

En la aplicación real, Flask se usa en create_app() para cargar configuración, inicializar extensiones, registrar blueprints y configurar manejadores de error. Los blueprints separan rutas de catálogo, autenticación, reseñas, biblioteca, perfil y administración.

Aunque Flask no obliga a usar MVC de forma estricta, el proyecto se ha organizado con una idea parecida: los modelos representan las tablas de base de datos, las rutas actúan como controladores y las plantillas Jinja2 hacen de vistas. Esta separación ayuda a que el código sea más fácil de leer y evita mezclar toda la lógica en un único archivo.

Evidencia en el proyecto: esta organización se ve en los blueprints de catálogo, autenticación, biblioteca, reseñas, perfil y administración. En la defensa se puede abrir create_app() y mostrar cómo se registran rutas y extensiones antes de arrancar la aplicación.

3.4 Base de datos relacional y ORM

PostgreSQL se ha utilizado como base de datos principal por ser una base relacional potente y muy extendida. Para no escribir SQL manual en cada operación, se usa SQLAlchemy mediante Flask-SQLAlchemy. El ORM permite trabajar con clases Python como User, Game, Review o UserLibrary, y a la vez mantiene relaciones, claves foráneas y restricciones.

El modelo es pequeño, pero suficiente para demostrar relaciones uno a muchos y restricciones de unicidad. Por ejemplo, un usuario puede tener muchas reseñas, un juego puede tener muchas reseñas y la tabla reviews impide que el mismo usuario tenga dos reseñas para el mismo juego. Esta regla se controla tanto en la base de datos como en la lógica de la ruta.

Evidencia en el proyecto: las restricciones de unicidad de reviews y user_library evitan duplicados por usuario y juego. Esta regla se comprueba en las pruebas de reseña única y biblioteca personal.

3.5 Frontend con Jinja2, Bootstrap y CSS propio

Jinja2 permite construir plantillas HTML con bloques, includes, condiciones y bucles. En F2P Catalog se utiliza un layout base y varios parciales reutilizables, como la barra de navegación, los mensajes flash, las tarjetas de juego, el pie de página, la paginación y el input CSRF. Esto reduce duplicación y hace que las pantallas compartan estructura.

Bootstrap se ha usado como base visual porque acelera la creación de formularios, grids, tarjetas, botones y componentes responsive (Bootstrap, s. f.). Encima se ha añadido un CSS propio con estética oscura inspirada en sitios de videojuegos. Se ha evitado hacer CSS global demasiado agresivo porque en una revisión se detectó que podía romper tablas, alertas o tarjetas fuera del contexto deseado.

Evidencia en el proyecto: los parciales game_card.html y csrf_input.html se reutilizan en varias pantallas. Esta decisión evitó corregir el mismo componente en tres archivos distintos cuando se ajustó la presentación de las tarjetas.

3.6 Seguridad básica en aplicaciones web

La seguridad no se ha tratado como un extra, sino como parte normal de la aplicación. Las contraseñas se guardan hasheadas con bcrypt, los formularios POST incluyen token CSRF, las rutas privadas usan login_required y las rutas de administración necesitan además admin_required. Estas medidas son básicas, pero importantes para una aplicación con usuarios.

Ejemplos concretos: /logout, /biblioteca/agregar/<game_id>, /juego/<game_id>/resena y /admin/actualizar-juegos son acciones POST protegidas con CSRF. Las rutas privadas usan login_required y el panel /admin/resenas exige admin_required.

También se validan los datos en el servidor, no solo en el HTML. Esto incluye email, username, longitud de contraseña, rating, texto de reseña y estado de biblioteca. La validación doble, en interfaz y backend, ayuda a mejorar la experiencia del usuario y protege mejor la integridad de los datos (OWASP Foundation, s. f.).

3.7 Contenedores y despliegue como parte del marco técnico

Docker se ha usado para que el entorno sea más repetible. En proyectos web, uno de los problemas habituales es que algo funcione en un ordenador y falle en otro por versiones distintas de Python, PostgreSQL o dependencias. Al usar contenedores se define una imagen para la aplicación y otra para la base de datos, lo que reduce esas diferencias.

Docker Compose permite levantar varios servicios con un solo comando. En este caso se levanta la aplicación web y PostgreSQL. Para producción ligera se usa otro compose con Gunicorn y reinicio automático. Es una arquitectura sencilla y adecuada para el alcance de clase: permite probar despliegue, variables de entorno, persistencia y logs sin montar una infraestructura empresarial.

En local se usa docker-compose.yml con Flask en modo desarrollo. En producción ligera se usa docker-compose.prod.yml con Gunicorn, 1 worker, 4 threads y el puerto 5000 enlazado solo a 127.0.0.1. En la memoria se usa la forma “docker compose” para el comando actual, aunque el archivo conserve el nombre docker-compose.yml.

Gunicorn se usa porque el servidor de desarrollo de Flask no debe utilizarse como servidor de producción. Esta diferencia fue importante para la memoria: desarrollar y desplegar no son exactamente lo mismo. En local se prioriza comodidad y recarga rápida; en producción se prioriza estabilidad y configuración más segura.

4. Análisis y planificación del proyecto

4.1 Usuarios y roles

Rol	Pantallas/rutas accesibles	Acciones principales
Visitante anónimo	/, /catalogo, /juego/<id>, /registro, /login	Consultar juegos, filtrar, ver fichas y leer reseñas públicas.
Usuario registrado	Rutas públicas + /mi-biblioteca, /perfil, reseñas y acciones de biblioteca	Gestionar biblioteca, crear/editar/eliminar sus reseñas y cerrar sesión por POST.
Administrador	Rutas de usuario + /admin/resenas y /admin/actualizar-juegos	Moderar reseñas y lanzar actualización manual del catálogo.

4.2 Requisitos funcionales

ID	Actor / prioridad / requisito	Criterio de aceptación / prueba
RF-01	Visitante · Alta. El sistema mostrará una página de inicio con juegos destacados.	La home carga juegos destacados o un fallback de juegos recientes. Prueba asociada: CP-01 o flujo por rol.
RF-02	Visitante · Alta. El sistema permitirá consultar un catálogo paginado.	El catálogo muestra 20 juegos por página y permite navegar. Prueba asociada: CP-02 o flujo por rol.
RF-03	Visitante · Alta. El sistema permitirá buscar por texto.	La búsqueda filtra por título o descripción breve. Prueba asociada: CP-03 o flujo por rol.
RF-04	Visitante · Alta. El sistema permitirá filtrar por género, plataforma y publisher.	Los filtros se mantienen al paginar. Prueba asociada: CP-04 o flujo por rol.
RF-05	Visitante · Alta. El sistema permitirá ver la ficha de un juego.	La ficha muestra datos, requisitos, capturas y reseñas. Prueba asociada: CP-05 o flujo por rol.
RF-06	Anónimo/Usuario · Alta. El sistema permitirá registrar usuarios.	Se valida username, email, contraseña y confirmación. Prueba asociada: CP-06 o flujo por rol.
RF-07	Anónimo/Usuario · Alta. El sistema permitirá iniciar y cerrar sesión.	Login correcto crea sesión; logout se realiza por POST. Prueba asociada: CP-07 o flujo por rol.
RF-08	Usuario · Media. El usuario podrá añadir juegos a su biblioteca.	No se permite duplicar el mismo juego en la biblioteca. Prueba asociada: CP-08 o flujo por rol.
RF-09	Usuario · Media. El usuario podrá	Solo acepta want_to_play, playing

ID	Actor / prioridad / requisito	Criterio de aceptación / prueba
	cambiar el estado de un juego guardado.	o played. Prueba asociada: CP-09 o flujo por rol.
RF-10	Usuario · Media. El usuario podrá quitar juegos de su biblioteca.	Solo puede quitar entradas propias. Prueba asociada: CP-10 o flujo por rol.
RF-11	Usuario · Media. El usuario podrá publicar una reseña.	Rating 1-5 y texto 10-1000 caracteres. Prueba asociada: CP-11 o flujo por rol.
RF-12	Usuario · Media. El usuario podrá editar o eliminar su reseña.	Solo el autor puede editar o eliminar desde la ruta de usuario. Prueba asociada: CP-12 o flujo por rol.
RF-13	Usuario · Media. El perfil mostrará estadísticas básicas.	Número de reseñas, biblioteca y últimas reseñas. Prueba asociada: CP-13 o flujo por rol.
RF-14	Admin · Media. El administrador podrá ver todas las reseñas.	El panel admin lista reseñas con usuario y juego. Prueba asociada: CP-14 o flujo por rol.
RF-15	Admin · Media. El administrador podrá eliminar reseñas.	La acción se hace por POST con CSRF y redirección segura. Prueba asociada: CP-14 o flujo por rol.
RF-16	Admin · Media. El administrador podrá actualizar el catálogo.	El sistema llama al seed de juegos y aplica cooldown. Prueba asociada: CP-14 o flujo por rol.

4.3 Requisitos no funcionales

ID	Categoría	Criterio medible o verificable
RNF-01	Usabilidad	Navegación principal visible y mensajes flash en operaciones importantes.
RNF-02	Responsive	Pantallas revisadas en escritorio y móvil usando Bootstrap 5.
RNF-03	Seguridad	POST con CSRF, contraseñas bcrypt y rutas privadas protegidas.
RNF-04	Mantenibilidad	Código separado en models, routes, services, templates y static.
RNF-05	Disponibilidad de datos	Catálogo servido desde PostgreSQL aunque no se consulte la API en cada visita.

ID	Categoría	Criterio medible o verificable
RNF-06	Rendimiento razonable	Catálogo paginado en servidor con 20 juegos por página.
RNF-07	Gestión de errores	404/500 personalizados y CSRFError con mensaje entendible.
RNF-08	Trazabilidad	Seeds muestran resumen processed/failed y juegos revisados/creados/actualizados.
RNF-09	Portabilidad	Entorno reproducible con Docker Compose y variables .env.
RNF-10	Privacidad	No se versionan secretos reales y solo se piden datos mínimos.

4.4 Historias de usuario

ID	Historia	Criterio
HU-01	Como visitante quiero ver juegos destacados para descubrir títulos al entrar.	La home muestra hasta 8 juegos.
HU-02	Como visitante quiero buscar juegos por texto para encontrar uno concreto.	El catálogo filtra por búsqueda.
HU-03	Como visitante quiero filtrar por género y plataforma para reducir resultados.	Los selects funcionan y mantienen estado.
HU-04	Como visitante quiero ver una ficha con requisitos y capturas para decidir si me interesa.	La ficha muestra detalles si existen.
HU-05	Como usuario quiero registrarme para poder usar funciones privadas.	Se crea cuenta con datos válidos.
HU-06	Como usuario quiero iniciar sesión para acceder a biblioteca y reseñas.	Se crea sesión con credenciales correctas.
HU-07	Como usuario quiero guardar un juego en mi biblioteca para recordarlo.	El juego queda en want_to_play.
HU-08	Como usuario quiero cambiar el estado de un juego para organizar mi progreso.	Se guarda el nuevo estado válido.
HU-09	Como usuario quiero escribir una reseña para compartir mi opinión.	Se publica si cumple rating y longitud.

ID	Historia	Criterio
HU-10	Como usuario quiero editar mi reseña si cambio de opinión.	El formulario aparece con valores previos.
HU-11	Como administrador quiero eliminar reseñas inadecuadas para moderar la web.	Se elimina por POST protegido.
HU-12	Como administrador quiero actualizar el catálogo para incorporar cambios de la API.	La actualización muestra resumen y respeta cooldown.

4.5 Casos de uso

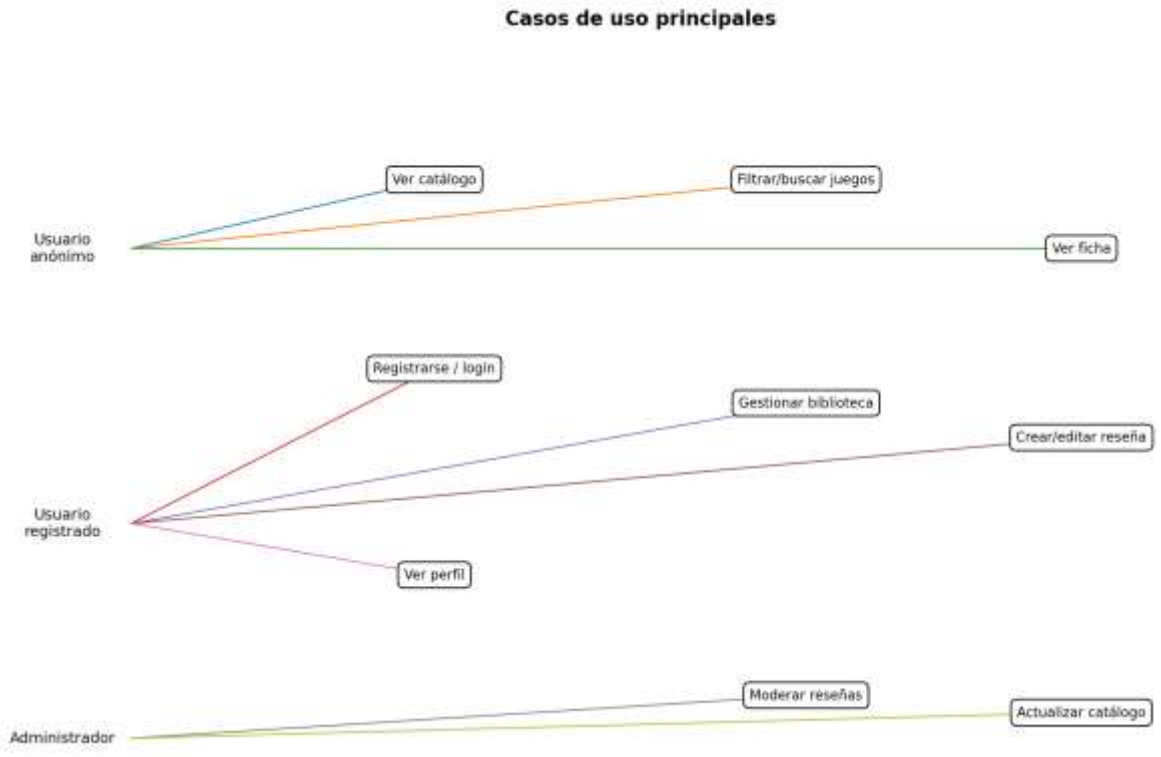


Figura 1. Casos de uso principales por rol de usuario.

Caso	Actor	Resumen
CU-01 Consultar catálogo	Anónimo / registrado	El usuario entra en /catalogo y navega por juegos paginados.
CU-02 Ver ficha	Anónimo / registrado	El usuario abre /juego/<id> y ve datos completos del juego.
CU-03 Registrarse	Anónimo	El usuario envía username, email, password y confirmación.
CU-04 Iniciar sesión	Anónimo	El usuario inicia sesión con email y contraseña.

Caso	Actor	Resumen
CU-05 Gestionar biblioteca	Registrado	Añade juego, cambia estado o lo elimina de su biblioteca.
CU-06 Gestionar reseña propia	Registrado	Crea, edita o elimina una reseña propia.
CU-07 Moderar reseñas	Administrador	Lista y borra reseñas de cualquier usuario.
CU-08 Actualizar catálogo	Administrador	Lanza seed_games para revisar datos desde FreeToGame.

4.6 Planificación

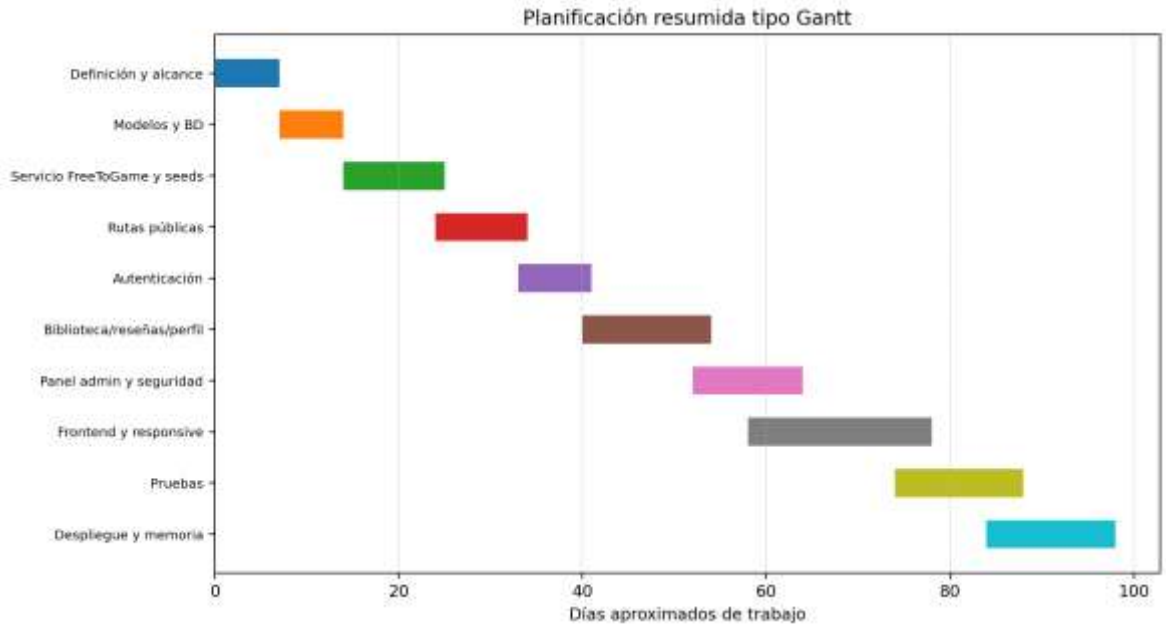


Figura 2. Planificación resumida del proyecto tipo Gantt.

Fase	Fechas aproximadas	Trabajo principal y reparto
0	17/02-25/02	Propuesta, tema, rúbrica y documentación inicial. Ambos.
1	05/04-06/04	Base Flask, configuración, Docker y arquitectura inicial. Pablo Laya + decisiones de ambos.
2	06/04	Modelos users, games, reviews y user_library. Pablo Laya.
3	07/04	Catálogo público, filtros, paginación y seeds. Pablo Laya.
4	07/04	Registro, login, logout y validaciones. Pablo Laya.

Fase	Fechas aproximadas	Trabajo principal y reparto
5	10/04-11/04	Biblioteca, reseñas, perfil, administración y hardening backend. Pablo Laya.
6	11/04-07/05	Frontend, tema visual, navegación, responsive y ajustes de UX. Pablo Pérez Lagares.
7	07/05-08/05	Despliegue, seed ampliado, pruebas manuales y memoria final. Despliegue: Pablo Laya; pruebas manuales: Pablo Pérez Lagares; revisión final: ambos.

4.7 Riesgos

Riesgo	Prob.	Impacto	Estado final / medida aplicada
Dependencia de API externa	Media	Alta	Controlado: catálogo cacheado en PostgreSQL y errores tratados en servicio.
Rate limit o lentitud de FreeToGame	Media	Media	Controlado: pausa entre detalles y actualización manual desde admin.
CSS global que rompa pantallas	Media	Media	Ocurrió parcialmente: se acotaron estilos y se respetó Bootstrap.
Formularios sin CSRF	Media	Alta	Controlado: partial csrf_input y CSRFProtect global.
Duplicados en reseñas/biblioteca	Media	Media	Controlado: UniqueConstraint y comprobación previa en rutas.
Problemas de despliegue/Gunicorn	Media	Alta	Ocurrió: se estabilizó con 1 worker y 4 threads.
Scope creep	Alta	Media	Controlado: se dejaron pagos, móvil, recomendaciones y E2E como fuera de alcance.
Falta de pruebas automatizadas	Media	Media	Pendiente asumido: se documentaron pruebas manuales y queda como mejora.

4.8 Priorización de funcionalidades

La prioridad se decidió pensando en qué era imprescindible para que la aplicación tuviera sentido. Primero se necesitaba que el catálogo funcionara con datos reales y cacheados. Después se añadieron usuarios, porque sin usuario no hay biblioteca ni reseñas propias. Por último se cerraron administración, frontend visual, pruebas y despliegue.

Durante el desarrollo se recortó la idea de integrar varias APIs externas. Se mantuvo FreeToGame porque ya ofrecía juegos, imágenes, requisitos y screenshots. Ese recorte dejó más tiempo para terminar biblioteca, reseñas, panel admin y despliegue.

Prioridad	Funcionalidades	Motivo
Alta	Catálogo, ficha, seed de juegos, base de datos y autenticación.	Sin estas partes no existe producto mínimo demostrable.
Media	Biblioteca, reseñas, perfil y panel admin.	Aportan valor y demuestran interacción con usuario.
Media	Frontend responsive y tema visual.	Mejora la presentación y facilita la defensa.
Baja / futura	Recomendaciones, integración con tiendas, tests E2E y estadísticas avanzadas.	Son útiles, pero no necesarias para el alcance de clase.

4.9 Organización del trabajo

La documentación del proyecto reparte la defensa en dos bloques: backend y arquitectura por un lado, y frontend, UX y pruebas por otro. Esa división es lógica porque permite explicar el proyecto desde dos puntos de vista complementarios. Aun así, las partes están conectadas y se tuvieron que revisar juntas, sobre todo formularios, rutas y variables Jinja.

La colaboración cruzada se hizo revisando partes del otro: el frontend se probó con rutas reales y tokens CSRF, y el backend se revisó desde los formularios para comprobar que los name de inputs, redirecciones y mensajes flash coincidían. Uno de los problemas resueltos en equipo fue que un cambio visual podía romper un POST si se eliminaba un hidden input o el csrf_input.html.

Área	Responsable principal	Evidencia o resultado
Planificación, investigación, fuentes, herramientas, arquitectura y tecnologías	Ambos alumnos	Propuesta inicial, decisiones técnicas, selección de stack y estructura general.
Backend y modelo de datos	Pablo Laya	Modelos SQLAlchemy, rutas Flask, seeds, servicio FreeToGame y reglas de dominio.
Seguridad	Pablo Laya	bcrypt, Flask-Login, CSRF, admin_required, validaciones y errores seguros.

Área	Responsable principal	Evidencia o resultado
Despliegue	Pablo Laya	Docker Compose, Gunicorn, PostgreSQL y procedimiento de producción ligera.
Frontend y experiencia de usuario	Pablo Pérez Lagares	Plantillas Jinja2, Bootstrap, CSS, navegación, estados visuales y responsive.
Pruebas manuales	Pablo Pérez Lagares	Recorridos por rol, formularios, validaciones, interfaz y comprobación de flujos.
Memoria y defensa	Ambos alumnos	Revisión de contenido, reparto de explicación y preparación de evidencias.

5. Diseño del sistema

5.1 Arquitectura general

La aplicación se diseñó como una aplicación Flask server-rendered. El navegador realiza peticiones GET o POST, Flask ejecuta la ruta correspondiente, consulta modelos de SQLAlchemy o servicios, y Jinja2 genera la respuesta HTML. No existe una API REST pública propia para el frontend; el propio backend renderiza las vistas. La API externa FreeToGame solo se consume desde el servidor para llenar o actualizar la base de datos.

Ejemplo de flujo real: al entrar en /catalogo, Flask ejecuta la ruta del blueprint de juegos, construye la consulta con filtros sobre PostgreSQL, pagina los resultados, envía las variables a games/catalog.html y Jinja2 devuelve el HTML final al navegador. Fuente de los diagramas: elaboración propia a partir del código del proyecto.

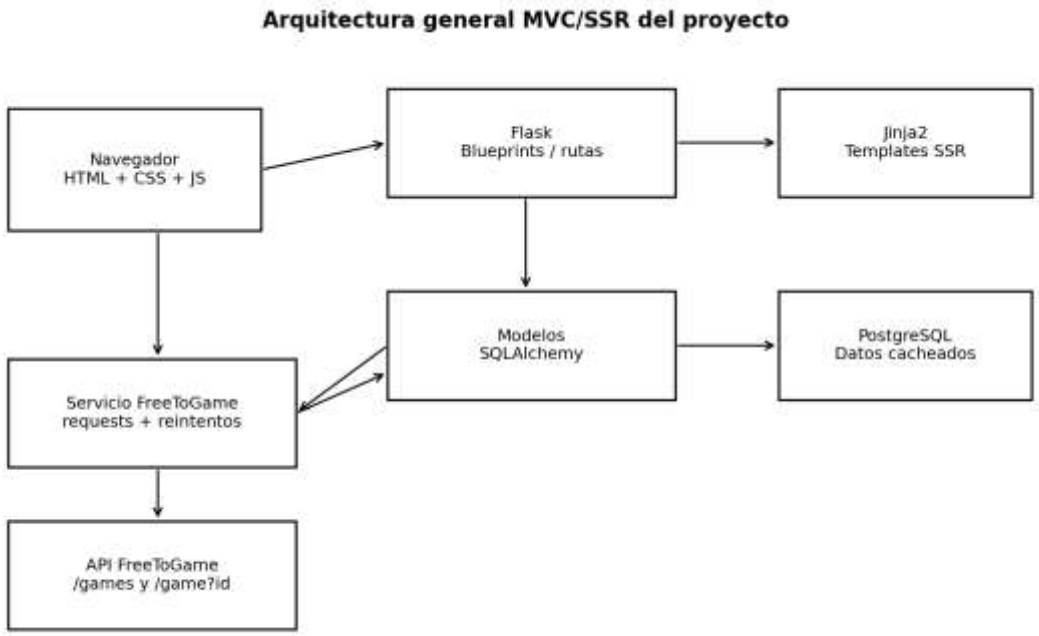


Figura 3. Arquitectura general MVC/SSR del proyecto.

Esta arquitectura es más sencilla que separar frontend y backend en dos proyectos distintos. Para el nivel del proyecto, ayuda a mantener todo en un repositorio y evita problemas añadidos de CORS, tokens JWT o hosting separado. Aun así, se mantiene separación interna mediante blueprints, modelos, servicios y plantillas.

5.2 Modelo de datos

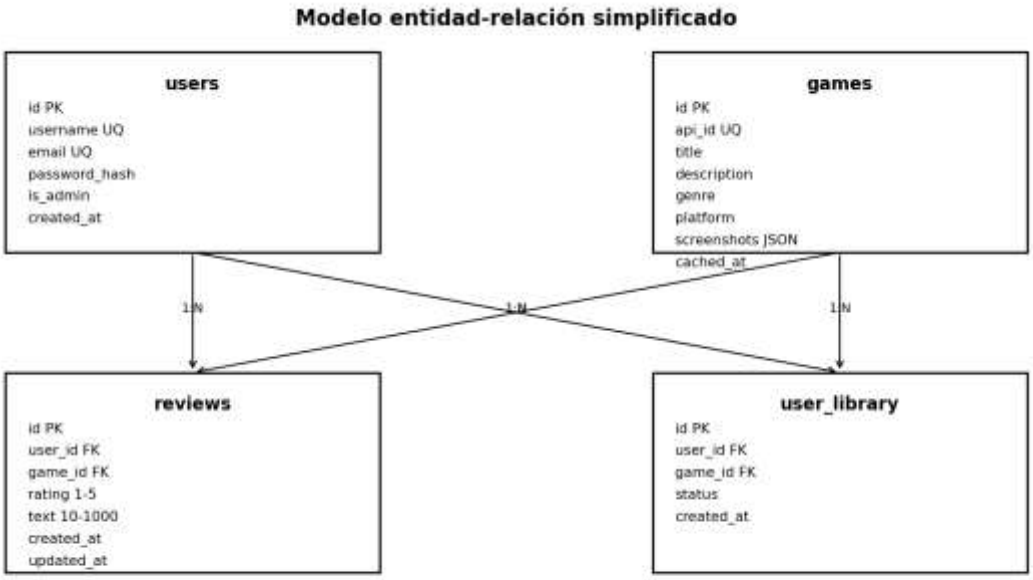


Figura 4. Modelo entidad-relación simplificado.

El modelo de datos contiene cuatro tablas principales. No se han creado tablas de roles porque solo existen dos niveles: usuario normal y administrador. Por eso se utiliza un booleano is_admin en la tabla users. Tampoco se ha usado una tabla extra para estados de biblioteca porque los estados son tres y se validan en el modelo y en las rutas.

5.3 Diccionario de datos

Tabla	Campo	Tipo / regla	Descripción
users	id	Integer PK	Identificador interno.
users	username	String(30), not null, unique	Nombre visible validado con regex 3-30.
users	email	String(255), not null, unique	Correo normalizado en minúsculas.
users	password_hash	String(255), not null	Hash bcrypt; nunca contraseña en claro.
users	is_admin	Boolean, default False	Rol administrador.
users	created_at	DateTime UTC	Alta de cuenta.
games	api_id	Integer, not null, unique	ID original de FreeToGame.
games	title/genre/platform/publisher/developer	String/Text	Datos principales del juego.

Tabla	Campo	Tipo / regla	Descripción
games	description	Text, not null	Descripción larga obligatoria.
games	req_os/processor/memory/graphics/storage	String nullable	Requisitos mínimos si la API los proporciona.
games	screenshots	JSON, default list	URLs de capturas.
games	cached_at	DateTime UTC	Última caché/actualización.
reviews	user_id/game_id	FK + unique conjunta	Una reseña por usuario y juego.
reviews	rating/text	Integer 1-5 / Text 10-1000	Valoración y texto obligatorios.
reviews	created_at/updated_at	DateTime UTC	Alta y última edición.
user_library	user_id/game_id	FK + unique conjunta	Un juego por usuario en biblioteca.
user_library	status	String validado	want_to_play, playing o played.
user_library	created_at	DateTime UTC	Fecha de guardado.

5.4 Diseño de rutas y endpoints

Como el proyecto es SSR, las rutas devuelven HTML renderizado, no JSON para un frontend desacoplado. Aun así, se documentan como endpoints HTTP porque forman parte del diseño de la aplicación.

Ruta	Método	Acceso	Uso / error esperado
/	GET	Público	Home con juegos destacados o fallback.
/catalogo	GET	Público	Catálogo con filtros, búsqueda, orden y paginación.
/juego/<id>	GET	Público	Ficha; ID inexistente devuelve 404.
/registro	GET/POST	Anónimo	Alta con validación de username, email y password.
/login	GET/POST	Anónimo	Inicio de sesión y validación de next local.
/logout	POST	Usuario	Cierre de sesión con CSRF.
/mi-biblioteca	GET	Usuario	Listado privado de

Ruta	Método	Acceso	Uso / error esperado
			biblioteca.
/biblioteca/agregar/<game_id>	POST	Usuario	Añadir juego; duplicado muestra aviso.
/biblioteca/estado/<id>	POST	Usuario	Cambiar estado propio; estado inválido se rechaza.
/biblioteca/quitar/<id>	POST	Usuario	Eliminar entrada propia.
/juego/<game_id>/resena	POST	Usuario	Crear reseña; rating/texto se validan.
/resena/<id>/editar	GET/POST	Autor	Editar reseña propia; ajena redirige con error.
/resena/<id>/eliminar	POST	Autor	Eliminar reseña propia con CSRF.
/perfil	GET	Usuario	Resumen de cuenta, biblioteca y reseñas.
/admin/resenas	GET	Admin	Moderación; usuario normal no accede.
/admin/resenas/<id>/eliminar	POST	Admin	Borrado admin con CSRF.
/admin/actualizar-juegos	POST	Admin	Actualización de catálogo con cooldown.

5.5 Uso de la API FreeToGame

La API externa se usa de forma controlada desde el backend. El endpoint /games devuelve el listado básico y /game?id=<id> devuelve el detalle. En el seed se compara si un juego ya existe y si su detalle está completo para evitar trabajo innecesario. Esta decisión reduce llamadas y hace que el refresco sea más eficiente.

Si FreeToGame falla, devuelve datos incompletos o tarda demasiado, el servicio no debe tirar la aplicación completa. El diseño intenta que el catálogo público siga funcionando con los datos ya cacheados y que el fallo se refleje en el resumen del seed o en un mensaje de administración.

Endpoint externo	Momento de uso	Datos aprovechados
/games	Seed inicial o actualización manual	id, title, thumbnail, genre, platform, developer, publisher, release_date y enlaces.
/game?id=<api_id>	Solo cuando hace falta detalle	description, status, requisitos mínimos y screenshots.

5.6 Diseño de interfaz y experiencia de usuario

Las capturas siguientes se incluyen como evidencia visual de la versión desplegada. Se han escogido tres pantallas públicas porque permiten comprobar el flujo básico sin usar credenciales: inicio, catálogo y ficha de juego. Las vistas privadas se documentan más adelante en el manual de usuario con usuario de demostración.

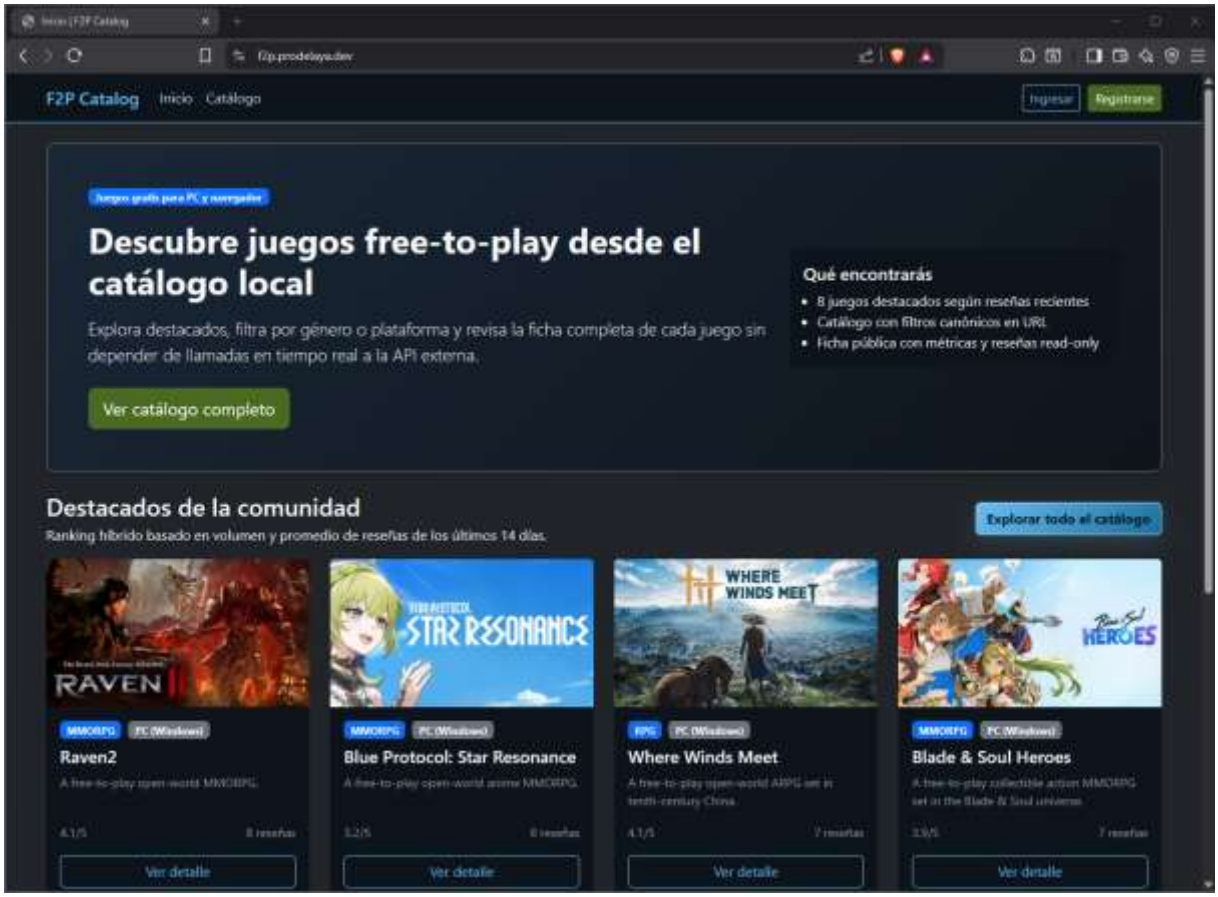


Figura 6. Página de inicio con hero, resumen del proyecto y juegos destacados. Fuente: elaboración propia a partir de la web desplegada.

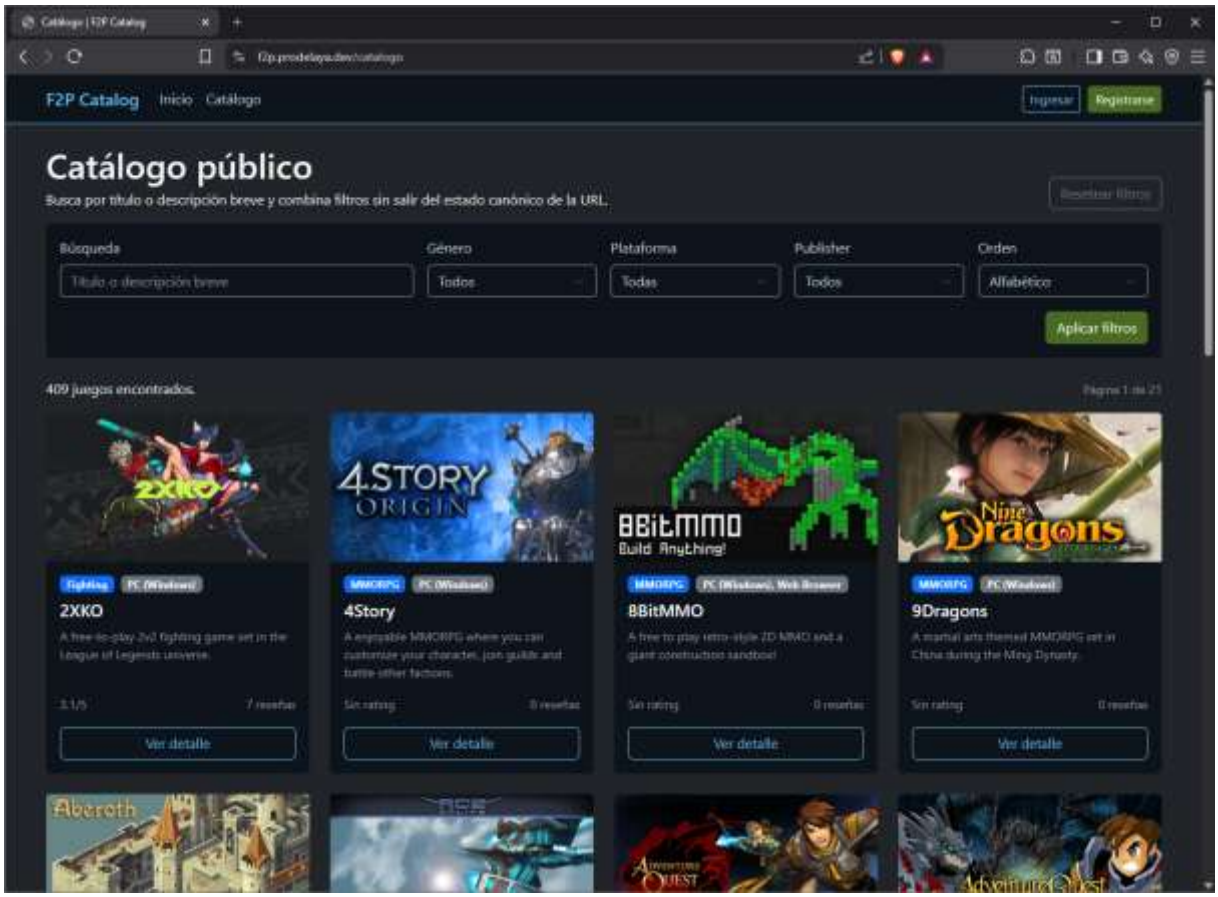


Figura 7. Catálogo público con filtros, ordenación, contador de resultados y paginación. Fuente: elaboración propia a partir de la web desplegada.

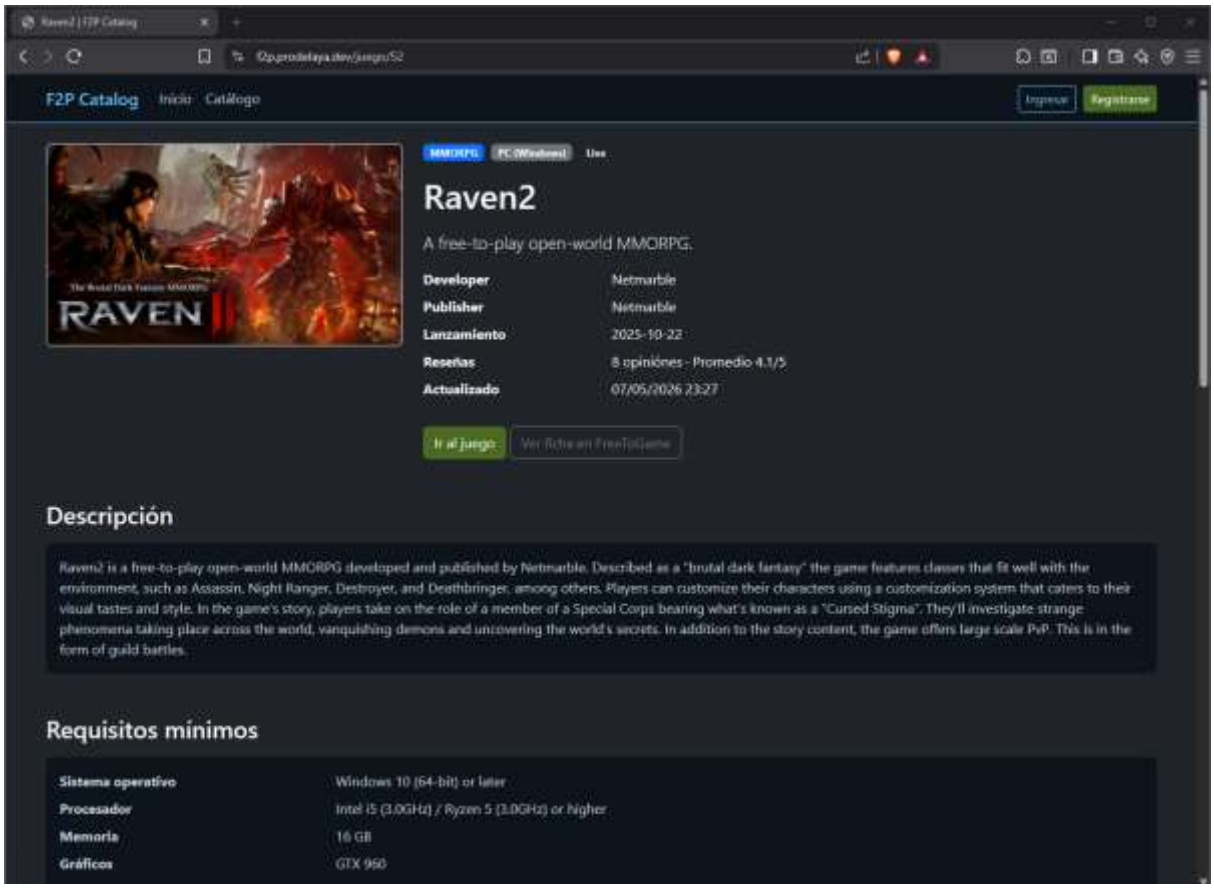


Figura 8.1 Ficha pública de Raven2 con datos del juego, requisitos mínimos, capturas y reseñas. Fuente: elaboración propia a partir de la web desplegada.

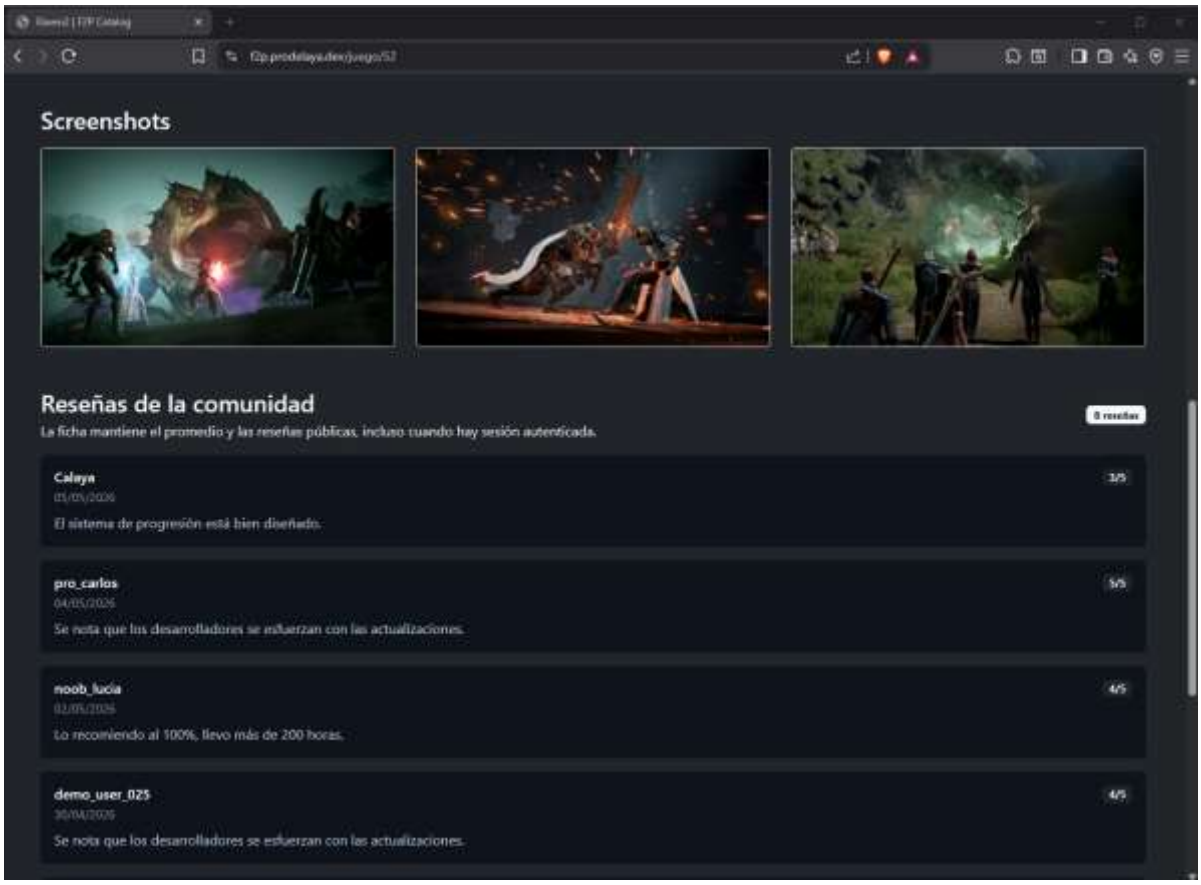


Figura 8.2 Ficha pública de Raven2 con datos del juego, requisitos mínimos, capturas y reseñas. Fuente: elaboración propia a partir de la web desplegada.

La interfaz se ha planteado con un tema oscuro de estilo gaming, pero sin perder claridad. La navegación principal muestra Inicio y Catálogo para todos. Si hay sesión, se muestran Mi biblioteca, Mi perfil mediante el nombre de usuario y Cerrar sesión. Si el usuario es administrador, aparece además Administración.

Las pantallas principales se han organizado con Bootstrap: grids para tarjetas de juego, cards para bloques de información, formularios con form-control y form-select, alertas para mensajes flash y tablas responsivas en administración. El objetivo no ha sido hacer una interfaz espectacular, sino una interfaz clara, estable y fácil de defender.

Pantalla	Objetivo UX	Detalles de diseño
Home	Presentar el proyecto y juegos destacados.	Hero simple, botón a catálogo y tarjetas reutilizables.
Catálogo	Encontrar juegos rápido.	Formulario GET con búsqueda, filtros, orden y paginación.
Ficha	Mostrar toda la información de un juego.	Datos, enlaces, descripción, requisitos, capturas, biblioteca y reseñas.
Login/registro	Entrar o crear cuenta sin confusión.	Errores visibles y campos validados.
Biblioteca	Organizar juegos guardados.	Filtros por estado y acciones por

Pantalla	Objetivo UX	Detalles de diseño
		entrada.
Admin	Moderar con rapidez.	Listado de reseñas, borrado y botón de actualización.

5.7 Contratos entre backend y frontend

Una parte importante del diseño fue tratar las plantillas como un contrato entre backend y frontend. Cada template recibe nombres concretos desde la ruta. Por ejemplo, catalog.html espera games, pagination, current_filters y filter_options. Si se cambian esos nombres o se eliminan hidden inputs de filtros, la página puede dejar de renderizar o perder la paginación.

Template	Variables relevantes	Riesgo si se cambia mal
games/catalog.html	games, pagination, current_filters, filter_options, total_count.	Se rompen filtros, paginación o contador.
games/detail.html	game, reviews, review_summary, own_review, library_entry.	Se rompen reseñas, biblioteca o ficha.
auth/login.html	form_data, errors, general_error, next_value.	Se pierden errores o redirección segura.
library/my_library.html	entries, current_status, available_statuses.	Se rompen filtros y acciones de biblioteca.
admin/reviews.html	reviews, has_results, refresh_status.	No se muestra cooldown o listado admin.

5.8 Estados vacíos y errores en la interfaz

El diseño también contempla estados vacíos. Si no hay reseñas, el usuario ve un mensaje; si la biblioteca está vacía, se le ofrece volver al catálogo; si no hay resultados con filtros, se muestra una alerta. Estos estados son pequeños, pero hacen que la aplicación parezca más cuidada y evitan pantallas en blanco.

6. Desarrollo e implementación

6.1 Entorno de desarrollo

Elemento	Uso en el proyecto
Python	Python 3.11 definido en Dockerfile (python:3.11-slim).
Flask y extensiones	Flask, Flask-SQLAlchemy, Flask-Login, Flask-Bcrypt y Flask-WTF en requirements.txt.
Base de datos	PostgreSQL 15 en docker-compose.yml y docker-compose.prod.yml.
ORM	SQLAlchemy a través de Flask-SQLAlchemy.
Frontend	Jinja2, Bootstrap 5 por CDN, CSS propio y JavaScript vanilla puntual.
Infraestructura	Docker Compose en local y producción ligera; Gunicorn en docker-compose.prod.yml.
Control de versiones	Git y GitHub: https://github.com/Prodelaya/Proyecto-DAW-Juegos-F2P.
Sistema probado	Desarrollo local con Docker; despliegue documentado en servidor Ubuntu.

6.2 Estructura del repositorio

```
f2p-catalog/
├── app/
│   ├── models/       # User, Game, Review, UserLibrary
│   ├── routes/       # blueprints de Flask
│   ├── services/     # cliente de FreeToGame
│   ├── templates/    # vistas Jinja2 y parciales
│   └── static/       # CSS, JS e imágenes
├── seeds/            # carga inicial y datos demo
├── scripts/          # espera de PostgreSQL
├── docs/             # documentación del proyecto
├── Dockerfile
├── docker-compose.yml
├── docker-compose.prod.yml
├── requirements.txt
└── README.md
```

La estructura está separada por responsabilidad. Esto ayuda a que el proyecto no dependa de un único archivo grande y hace más fácil localizar una funcionalidad concreta. Por ejemplo, todo lo relacionado con la biblioteca está en el modelo UserLibrary, en routes/library.py y en templates/library/my_library.html.

Repositorio: https://github.com/Prodelaya/Proyecto-DAW-Juegos-F2P.

6.3 Factoría de aplicación y extensiones

El proyecto usa el patrón Application Factory. La función create_app() crea la aplicación, carga configuración, inicializa extensiones, registra filtros, importa modelos, crea tablas y registra rutas. Esta forma es algo más ordenada que crear una app global desde el principio.

```
def create_app():
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object(Config)

    db.init_app(app)
    login_manager.init_app(app)
    bcrypt.init_app(app)
    csrf.init_app(app)

    app.jinja_env.filters["madrid_datetime"] = format_madrid_datetime
    register_routes(app)
    return app
```

Las extensiones se instancian en extensions.py para evitar importaciones circulares. Esta decisión se nota sobre todo en los modelos y rutas, porque todos pueden importar db, bcrypt, login_manager o csrf desde el mismo sitio.

6.4 Modelos y validaciones

Una decisión importante fue poner validaciones también en los modelos. Por ejemplo, el username se valida con una expresión regular, la reseña exige rating entre 1 y 5, y el texto debe tener entre 10 y 1000 caracteres. No se confía solo en el HTML, ya que un usuario podría enviar una petición manipulada.

```
@validates("rating")
def validate_rating(self, _key, rating: int) -> int:
    if type(rating) is not int or not 1 <= rating <= 5:
        raise ValueError("La valoración debe estar entre 1 y 5.")
    return rating
```

La tabla user_library usa un String para el estado en vez de un Enum de base de datos. Es una solución más simple para el alcance de clase, pero se controla con una lista cerrada de valores válidos. Para un proyecto más grande, podría valorarse un Enum o una tabla auxiliar.

6.5 Catálogo, filtros y paginación

El catálogo combina filtros opcionales con agregados de reseñas. Se permite buscar por texto, filtrar por género, plataforma y publisher, y ordenar por alfabético, número de reseñas,

valoración media o fecha de lanzamiento. La paginación se hace en servidor con 20 juegos por página.

```
if current_filters["q"]:
    search_term = f"%{current_filters['q']}%"
    query = query.filter(or_(
        Game.title.ilike(search_term),
        Game.short_description.ilike(search_term),
    ))

pagination = query.paginate(page=requested_page,
                            per_page=CATALOG_PER_PAGE,
                            error_out=False)
```

También se sanea la página solicitada. Si llega un valor raro o una página fuera de rango, la aplicación evita romperse y vuelve a un valor válido. Este tipo de detalle no se ve mucho desde fuera, pero ayuda bastante a que la aplicación sea estable.

6.6 Autenticación y sesiones

El registro valida campos, comprueba duplicados y guarda la contraseña mediante bcrypt. El login busca por email y llama a check_password. Si las credenciales son válidas se crea sesión con Flask-Login. El logout se realiza mediante POST, no mediante GET, porque cerrar sesión cambia el estado del usuario.

```
user = User.query.filter_by(email=form_data["email"]).first()
if user is None or not user.check_password(password):
    general_error = "Credenciales no válidas."
    return render_template("auth/login.html", ...)

login_user(user)
return redirect(_resolve_next_target())
```

La función que resuelve next comprueba que la URL sea local. Esto evita redirecciones abiertas, que podrían usarse para mandar al usuario a una página externa después del login.

6.7 Biblioteca y reseñas

La biblioteca personal permite añadir un juego y cambiar su estado. Para evitar modificaciones ajenas, antes de actualizar o borrar una entrada se comprueba que user_id coincide con el usuario actual. Las reseñas siguen una idea parecida: solo el autor puede editar o eliminar su reseña desde las rutas de usuario.

En el caso del administrador, la eliminación de reseñas se hace desde rutas admin protegidas. En la ficha de juego se muestra una papelera si el usuario autenticado es administrador, y esa acción va a admin_bp.delete_review mediante POST y con token CSRF.

6.8 Seed y sincronización con FreeToGame

El seed de juegos fue una de las partes más importantes del backend. En la primera carga puede tardar porque se consulta el listado y después el detalle de cada juego. Más adelante se mejoró para no pedir detalle si el juego no cambió y el dato local está completo.

```
needs_detail = is_new_game
if not needs_detail and game is not None:
    needs_detail = _has_basic_changes(game, basic_fields) or not _has_complete_local_detail(game)

if not needs_detail and game is not None:
    summary["unchanged"] += 1
    continue
```

Los seeds de usuarios, reseñas y biblioteca son idempotentes. Esto significa que se pueden ejecutar varias veces sin duplicar datos. Para la demo se crean usuarios normales, un administrador y datos de ejemplo suficientes para que la aplicación no se vea vacía.

6.9 Frontend y plantillas

El frontend se construyó con Jinja2 y Bootstrap. Durante el trabajo visual se mantuvieron los nombres de variables Jinja, los name de inputs, las llamadas url_for, los métodos POST, los tokens CSRF y los hidden inputs. Esto fue importante porque al mejorar una pantalla era fácil romper un formulario sin darse cuenta.

Se usan parciales para evitar repetir HTML. Por ejemplo, game_card.html se reutiliza en home, catálogo y biblioteca. También se usa csrf_input.html en todos los formularios POST. Esta solución es simple, pero ayuda mucho a mantener la coherencia del proyecto.

Ejemplos reales: partials/game_card.html se reutiliza en home, catálogo y biblioteca; partials/csrf_input.html se incluye en formularios POST; static/js/app.js se reserva para interacción puntual, sin convertir el proyecto en una SPA.

6.10 Decisiones técnicas destacadas

Decisión	Alternativa descartada	Motivo
Flask SSR/Jinja2	SPA React/Vue + API REST propia	Menor complejidad y mejor ajuste al alcance.
Cache en PostgreSQL	Consultar FreeToGame en cada visita	Más estable y permite filtros/paginación locales.
db.create_all()	Migraciones con Alembic	Simplificación del versión inicial; migraciones quedan como mejora.
is_admin booleano	Sistema RBAC con tabla de roles	Solo hay usuario normal y administrador.
CSRF en todos los POST	Proteger solo formularios críticos	Mejor criterio de seguridad y consistencia.

Decisión	Alternativa descartada	Motivo
Docker Compose	Instalación manual de servicios	Entorno más reproducible para defensa.
Gunicorn en producción	Servidor de desarrollo Flask	Flask run no es adecuado para producción.
No usar Vercel	Despliegue serverless/frontend estático	La app es Flask SSR con PostgreSQL.

6.11 Fechas, zona horaria y presentación de datos

Los timestamps se guardan en UTC. Esta decisión evita mezclar zonas horarias en la base de datos. Para el usuario, las fechas se muestran en hora de Madrid mediante el filtro Jinja madrid_datetime. Aunque pueda parecer un detalle menor, ayuda a separar persistencia y presentación.

Campos con fechas: users.created_at, games.cached_at, reviews.created_at, reviews.updated_at y user_library.created_at. La visualización se centraliza con el filtro Jinja madrid_datetime para mostrar hora de Madrid cuando corresponde.

```
def format_madrid_datetime(value, date_format="%d/%m/%Y %H:%M"):
    utc_value = value.replace(tzinfo=timezone.utc) if value.tzinfo is None else value
    return utc_value.astimezone(MADRID_TIMEZONE).strftime(date_format)
```

6.12 Manejo de errores

El proyecto incluye páginas 404 y 500 con el mismo layout que el resto de la web. Además, el error CSRF se captura para mostrar un mensaje entendible y redirigir a una página segura. En operaciones de escritura se hace rollback cuando SQLAlchemy lanza un error, para no dejar la sesión de base de datos en estado incorrecto.

Esta forma de gestionar errores es sencilla, pero mejora mucho la experiencia. En vez de mostrar una traza técnica, el usuario recibe un mensaje claro. Al mismo tiempo, el desarrollador puede revisar logs para entender qué ha fallado.

7. Pruebas y validación

7.1 Enfoque de pruebas

Las pruebas del proyecto se han planteado de forma manual y documentada. No se implementó una suite automatizada completa porque se priorizó cerrar una versión operativa y desplegable. Aun así, se probaron rutas públicas, formularios, validaciones, sesiones, permisos, semillas y despliegue local. La falta de tests automatizados se reconoce como una limitación y se incluye en las líneas futuras.

Tipo de prueba	Herramienta / forma	Objetivo
Funcional	Navegador	Comprobar que los flujos de usuario funcionan.
Validación	Formularios con datos incorrectos	Verificar mensajes y rechazo de entradas inválidas.
Seguridad básica	Peticiones POST y rutas privadas	Comprobar CSRF, login_required y admin_required.
Datos	Ejecución de seeds y revisión de listados	Confirmar que no se duplican datos y que se cachean juegos.
Despliegue local	Docker Compose y logs	Levantar web y base de datos desde cero.

7.2 Casos de prueba

ID	Caso	Entorno / fecha	Resultado obtenido
CP-01	Home y catálogo público	Local Docker, mayo 2026	Carga home, catálogo paginado y ficha. OK.
CP-02	Búsqueda y filtros	Local Docker, mayo 2026	Filtra por texto/género/plataforma y mantiene paginación. OK.
CP-03	Página inválida y ficha inexistente	Local Docker, mayo 2026	page=abc no rompe; ID inexistente muestra 404. OK.
CP-04	Registro válido e inválido	Navegador, mayo 2026	Valida email, password y duplicados. OK.
CP-05	Login válido e inválido	Navegador, mayo 2026	Credenciales erróneas no crean sesión; next externo no se acepta. OK.
CP-06	Logout	Navegador, mayo 2026	Solo por POST con CSRF. OK.
CP-07	Biblioteca:	Usuario demo, mayo	Crea entrada, cambia

ID	Caso	Entorno / fecha	Resultado obtenido
	añadir/cambiar/quitar	2026	estado y evita duplicados. OK.
CP-08	Reseñas: crear/editar/eliminar	Usuario demo, mayo 2026	Valida rating 1-5 y texto 10-1000. OK.
CP-09	Editar reseña ajena	Dos usuarios demo, mayo 2026	El usuario no puede modificar reseñas ajenas. OK.
CP-10	Panel admin con usuario normal	Usuario no admin, mayo 2026	Redirige o muestra aviso de permisos. OK.
CP-11	Panel admin con admin	Admin demo, mayo 2026	Lista reseñas, permite borrar y actualizar catálogo. OK.
CP-12	CSRF ausente/caducado	Petición manipulada, mayo 2026	Se rechaza y muestra aviso de formulario. OK.
CP-13	Seed completo repetido	Docker, mayo 2026	No duplica datos y muestra resumen processed/failed. OK.
CP-14	Arranque Docker	Docker Compose, mayo 2026	web y db levantan; wait_for_postgres evita carrera inicial. OK.

7.3 Resultados y observaciones

Incidencia	Qué se detectó	Corrección aplicada
INC-01	CSS global afectaba tarjetas y tablas	Se acotaron selectores y se revisaron tarjetas, alertas y tablas en home, catálogo y biblioteca.
INC-02	Gunicorn con varios workers duplicaba operaciones en la demo	Se dejó 1 worker y 4 threads para estabilizar el flujo de seed y formularios durante la defensa.
INC-03	Seed inicial lento por consulta de detalles	Se añadió resumen de creados, actualizados, sin cambios y fallidos para que el administrador supiera qué había ocurrido.

Tabla 7. Incidencias reales detectadas durante las pruebas manuales.

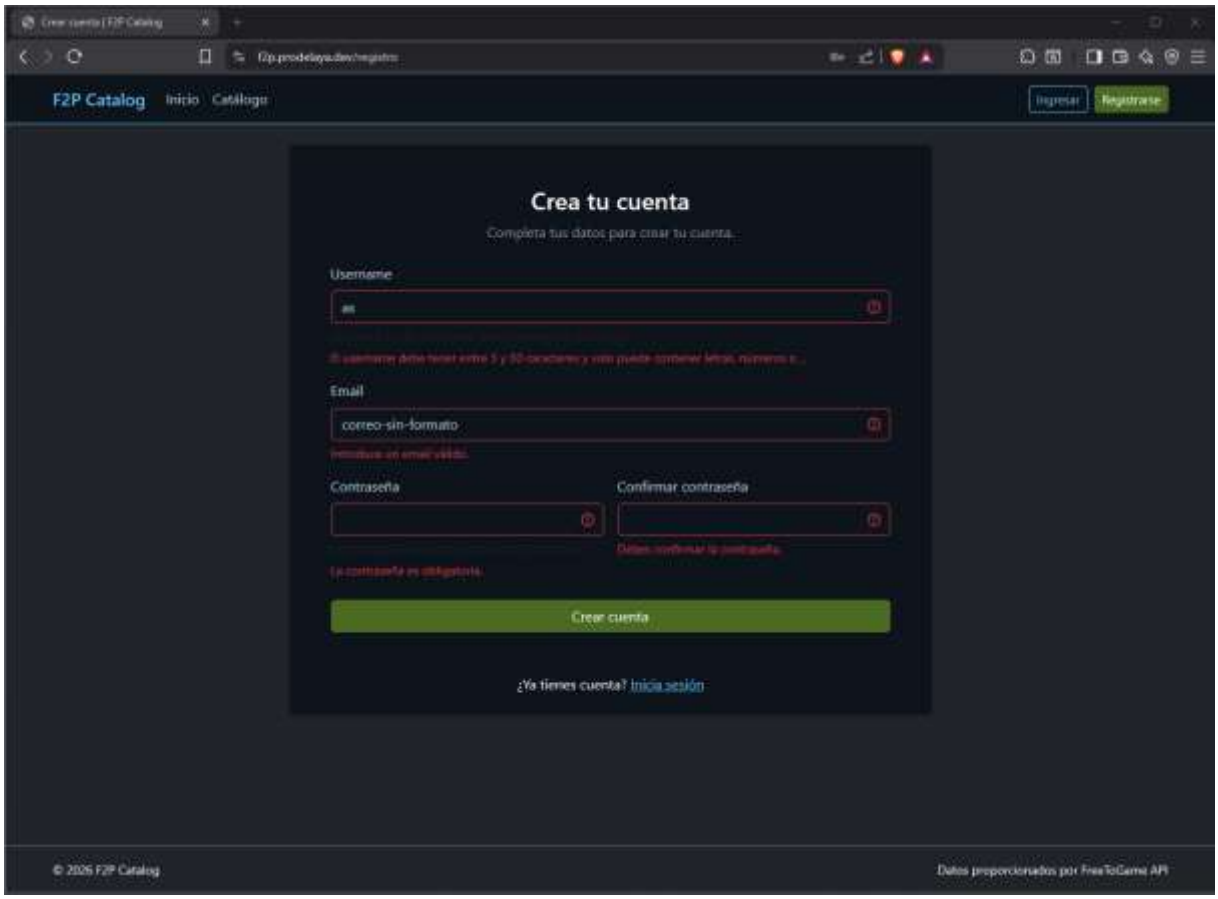


Figura 9. Prueba de validación de formulario con datos incorrectos y respuesta esperada. Fuente: elaboración propia.

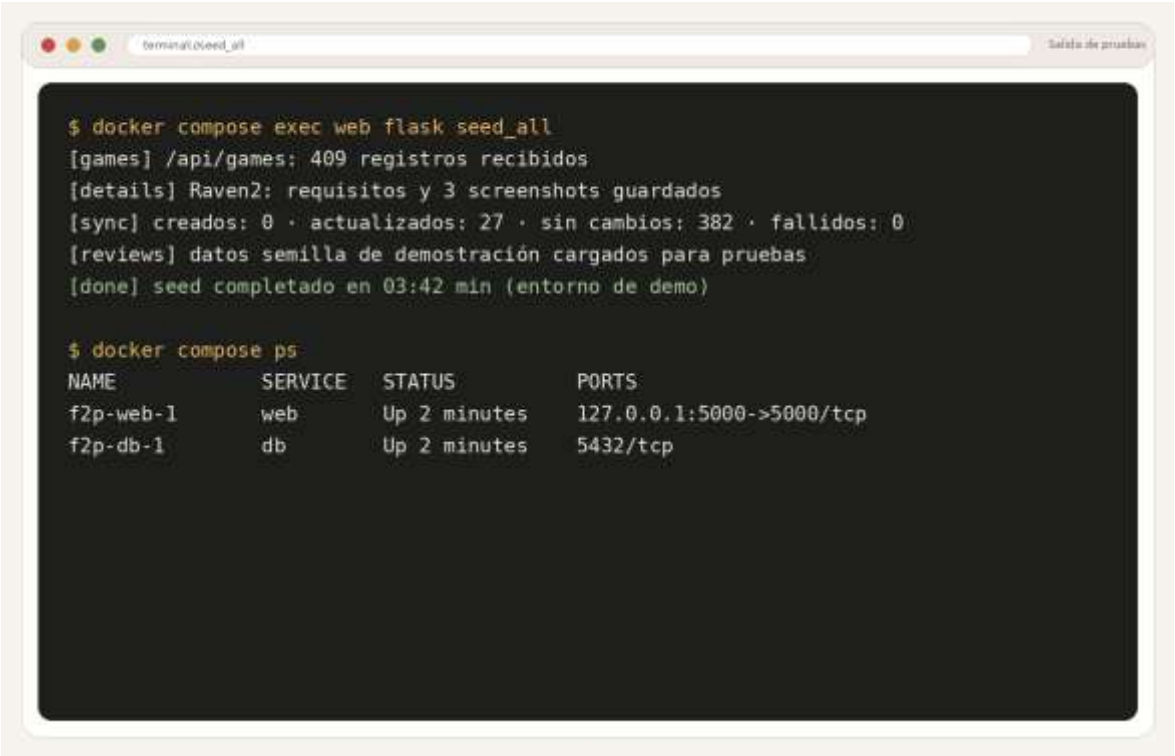


Figura 10. Salida resumida de seed y docker compose ps usada como evidencia técnica. Fuente: elaboración propia.

Las pruebas manuales permitieron detectar problemas reales. En frontend se observaron estilos globales que podían afectar zonas no previstas, por lo que se acotaron reglas CSS y se revisaron tarjetas, alertas y tablas. En despliegue se detectó que la configuración de Gunicorn con varios workers podía provocar una condición de carrera; se corrigió usando 1 worker y 4 threads para la demo.

También se detectó que la actualización del catálogo podía ser una operación lenta. Para mejorar la experiencia, se preparó un estado visual de “Actualizando catálogo...” y se devolvió un resumen de juegos revisados, creados, actualizados, sin cambios y fallidos. Esto ayuda a entender qué ha pasado después de pulsar el botón.

En despliegue se detectó una condición de carrera usando dos workers de Gunicorn. Para estabilizar el sistema se dejó un worker con cuatro threads. No es una solución de alta disponibilidad, pero sí una decisión razonable para el alcance del proyecto y la demo.

7.4 Pruebas por rol

Además de probar pantallas sueltas, se recorrieron flujos completos por rol. Esto es útil porque una ruta puede funcionar sola, pero fallar cuando se llega desde otra pantalla con parámetros, sesión o redirección next.

Rol	Flujo probado	Resultado
Anónimo	Inicio -> catálogo -> filtros -> ficha -> intento de acción privada.	Puede ver datos públicos y se le redirige a login en acciones privadas.
Registrado	Login -> ficha -> biblioteca -> reseña -> perfil -> logout.	Puede gestionar datos propios y no acceder a administración.
Administrador	Login admin -> panel -> borrar reseña -> actualizar catálogo.	Puede moderar y actualizar con cooldown.
Usuario malicioso simulado	Manipular status, page, rating o next.	El servidor sanea o rechaza valores inválidos.

7.5 Evidencias aportadas para la defensa

Las evidencias que acompañan la defensa son capturas de home, catálogo filtrado, ficha de juego, login/registro, biblioteca, reseña, panel admin, validaciones de formulario y salida de seed. En el vídeo se puede complementar con el historial de commits para justificar la evolución del proyecto y el reparto de tareas.

8. Despliegue y puesta en producción

8.1 Despliegue local con Docker Compose

El entorno local está preparado para levantarse con Docker Compose. El servicio web construye la imagen desde el Dockerfile, carga variables desde .env y escucha en el puerto 5000. El servicio db usa PostgreSQL 15 con un volumen persistente llamado pgdata.

Comprobaciones recomendadas: docker compose ps para ver web y db, docker compose logs -f web para revisar errores y abrir http://localhost:5000 para comprobar la interfaz.

```
cp .env.example .env
docker compose up --build
docker compose exec web python seeds/seed_all.py
```

El script wait_for_postgres.py espera a que PostgreSQL acepte conexiones antes de iniciar Flask. Esto evita que la aplicación arranque demasiado pronto y falle por no encontrar la base de datos lista.

8.2 Despliegue de producción

La URL pública comprobada para la entrega es https://f2p.prodelaya.dev/. La captura de inicio con barra de dirección visible se usa como evidencia de que el proyecto no se documenta solo en local.

Figura 11. Evidencia de despliegue público con URL visible en la barra del navegador. Fuente: elaboración propia a partir de la web desplegada.

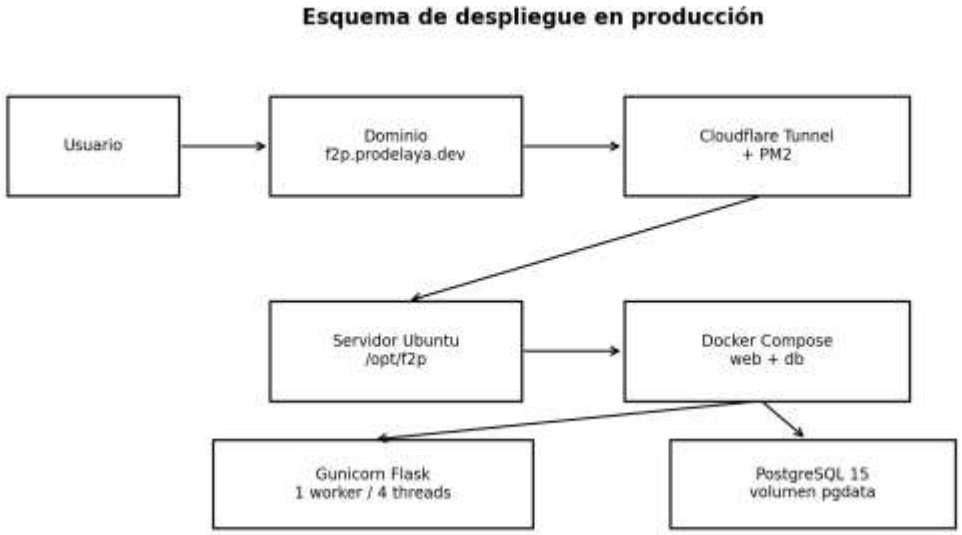


Figura 5. Esquema de despliegue en producción.

Para producción se documentó un despliegue en servidor Ubuntu bajo la ruta /opt/f2p. La aplicación se ejecuta con Gunicorn, PostgreSQL queda en Docker y la publicación externa se realiza mediante Cloudflare Tunnel gestionado con PM2. La URL pública documentada para la demo es https://f2p.prodelaya.dev/.

Antes de la defensa conviene comprobar que la URL pública está activa, que no muestra secretos y que el túnel o proxy apunta al servicio local correcto.

Componente	Decisión aplicada
Servidor	Ubuntu, ruta /opt/f2p.
Runtime	Gunicorn ejecutando app:create_app().
Workers	1 worker y 4 threads por estabilidad detectada.
Base de datos	PostgreSQL 15 en contenedor Docker.
Publicación	Cloudflare Tunnel gestionado con PM2.
Orquestación	docker-compose.prod.yml.
Puerto	5000 expuesto solo en 127.0.0.1 en producción.

8.3 Variables de entorno

Las variables sensibles no se versionan con valores reales. El repositorio incluye plantillas .env.example y .env.production.example para indicar qué se necesita configurar. En producción se debe usar una SECRET_KEY segura, credenciales de PostgreSQL no triviales y contraseña de administrador fuerte.

```
DATABASE_URL=postgresql://usuario:password@db:5432/f2p_catalog
SECRET_KEY=valor-largo-y-aleatorio
FLASK_ENV=production
```

```
FREETOGAME_API_URL=https://www.freetogame.com/api
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=password-segura
POSTGRES_USER=usuario
POSTGRES_PASSWORD=password-segura
POSTGRES_DB=f2p_catalog
```

8.4 Puesta en marcha en producción

1. Copiar el código del proyecto al servidor, por ejemplo en /opt/f2p.
2. Crear el archivo .env de producción con variables reales y seguras.
3. Levantar los servicios con docker compose -f docker-compose.prod.yml up -d --build.
4. Comprobar contenedores con docker compose -f docker-compose.prod.yml ps.
5. Ejecutar seeds si la base de datos está vacía.
6. Revisar logs del servicio web.
7. Configurar y arrancar Cloudflare Tunnel para publicar el puerto local.
8. Probar la URL pública, login, catálogo, biblioteca, reseñas y panel admin.

```
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml exec web python seeds/seed_all.py
docker compose -f docker-compose.prod.yml logs -f web
```

8.5 Copias de seguridad y mantenimiento

Para un proyecto de clase basta con un sistema básico, pero concreto. Antes de actualizar producción se propone realizar una copia manual con pg_dump y conservarla fuera del contenedor. Una periodicidad razonable para la demo sería una copia antes de cada despliegue importante y otra al cierre del proyecto.

Ejemplo: docker compose -f docker-compose.prod.yml exec db pg_dump -U $POSTGRES_USER $POSTGRES_DB > backup_f2p_catalog.sql

8.6 Checklist de despliegue

Comprobación	Descripción
Variables	SECRET_KEY, DATABASE_URL y credenciales no deben ser valores por defecto.
Contenedores	docker compose ps debe mostrar web y db levantados.
Logs	docker compose logs -f web no debe mostrar errores repetidos.
Base de datos	Seeds terminan con resumen y sin fallos importantes.
Catálogo	La URL pública o local muestra juegos y filtros funcionando.
Auth	Login y logout funcionan con CSRF.

Comprobación	Descripción
Admin	El panel solo abre con usuario administrador.
Backup	Existe copia pg_dump antes de cambios importantes.

9. Seguridad y protección de datos

9.1 Medidas técnicas aplicadas

Medida	Aplicación concreta	Prueba o riesgo que reduce
Hash de contraseñas	User.set_password/check_password con Flask-Bcrypt.	Evita guardar contraseñas en claro.
CSRF	csrf_input en logout, biblioteca, reseñas y admin.	CP-12: POST sin token rechazado.
Sesión	login_required en biblioteca, perfil y reseñas.	Anónimo redirigido a login.
Rol admin	admin_required en /admin/resenas y /admin/actualizar-juegos.	CP-10: usuario normal no accede.
Validación server-side	Auth, reviews, library y modelos.	Rating, status, email y texto inválidos se rechazan.
Redirecciones seguras	Validación de next local en login.	Evita open redirect.
ORM	SQLAlchemy para consultas.	Reduce riesgo de SQLi manual.
Errores personalizados	Handlers 404/500 y CSRFError.	Evita trazas técnicas al usuario.
Secretos en .env	Plantillas .env.example y .env.production.example.	No exponer credenciales reales.

9.2 Validación y control de datos

La validación se repite en varios niveles. El HTML ayuda al usuario con required, minlength o type=email, pero el backend vuelve a comprobar todo. Esta repetición puede parecer pesada, pero evita confiar en el navegador. Los modelos también validan reglas importantes para que la integridad no dependa solo de una ruta concreta.

Jinja2 escapa por defecto el contenido mostrado en plantillas, lo que ayuda frente a XSS básico. Aun así, se evita marcar texto de usuario como HTML seguro. Las reseñas se muestran como texto y no como contenido HTML.

9.3 Protección de datos y RGPD básico

La aplicación trata pocos datos personales. Principalmente se guarda username, email, hash de contraseña, reseñas y juegos añadidos a biblioteca. La finalidad es permitir autenticación, personalización y participación en el catálogo. No se tratan datos especialmente sensibles, pagos ni documentos oficiales.

Dato	Finalidad	Medida de protección
Email	Identificar cuenta e iniciar sesión.	Único, normalizado y no mostrado públicamente.

Dato	Finalidad	Medida de protección
Username	Nombre visible en perfil y reseñas.	Validado y sin datos obligatorios reales.
Password hash	Autenticación.	Bcrypt; no se guarda contraseña en claro.
Reseñas	Opinión pública del usuario.	Edición/eliminación por autor y moderación admin.
Biblioteca	Organización personal de juegos.	Solo visible y gestionable por el usuario.

Desde un punto de vista RGPD, habría que informar al usuario de qué datos se guardan, para qué se usan y cómo puede solicitar acceso, rectificación o supresión (Unión Europea, 2016). En la versión entregada se ha identificado esta necesidad, pero no se debe afirmar que el cumplimiento legal esté completo porque no aparece una política de privacidad visible en registro o footer.

Para una entrega real debería añadirse un texto visible de privacidad en registro o pie de página: qué datos se guardan, finalidad, responsable del proyecto académico y forma de solicitar eliminación de cuenta.

9.4 Limitaciones de seguridad

No se ha implementado verificación de email, recuperación de contraseña, rate limit avanzado de login ni auditoría completa de acciones admin. Los cambios mínimos antes de una producción real serían: añadir enlace de privacidad en registro y footer, permitir borrado de cuenta o anonimización de reseñas, aplicar rate limit en login y preparar recuperación de contraseña.

9.5 Buenas prácticas pendientes

Mejora pendiente	Por qué sería útil
Rate limit de login	Reducir intentos repetidos de contraseña.
Recuperación de contraseña	Permitir recuperar acceso sin intervención manual.
Verificación de email	Evitar cuentas con correos falsos.
Auditoría admin	Registrar quién elimina reseñas o actualiza catálogo.
Política de privacidad visible	Informar formalmente del tratamiento de datos.
Backups automáticos	Reducir pérdida de datos si falla el servidor.

10. Manual de usuario

10.1 Acceso a la aplicación

La entrada a la aplicación se realiza desde la URL pública o desde http://localhost:5000 en entorno local. Las capturas del manual se han preparado con datos de demostración y no muestran contraseñas ni variables sensibles.

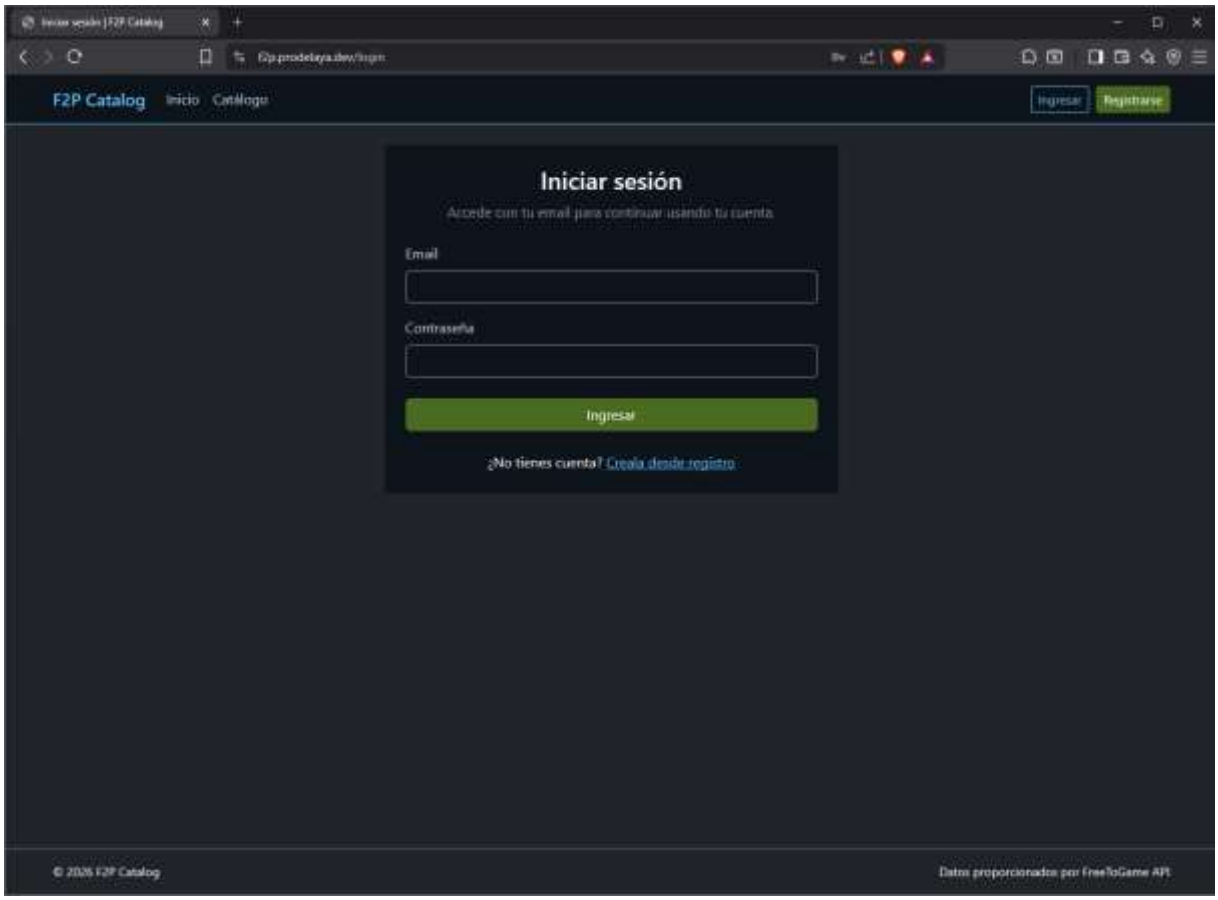


Figura 12. Pantallas de inicio de sesión con textos revisados. Fuente: elaboración propia.

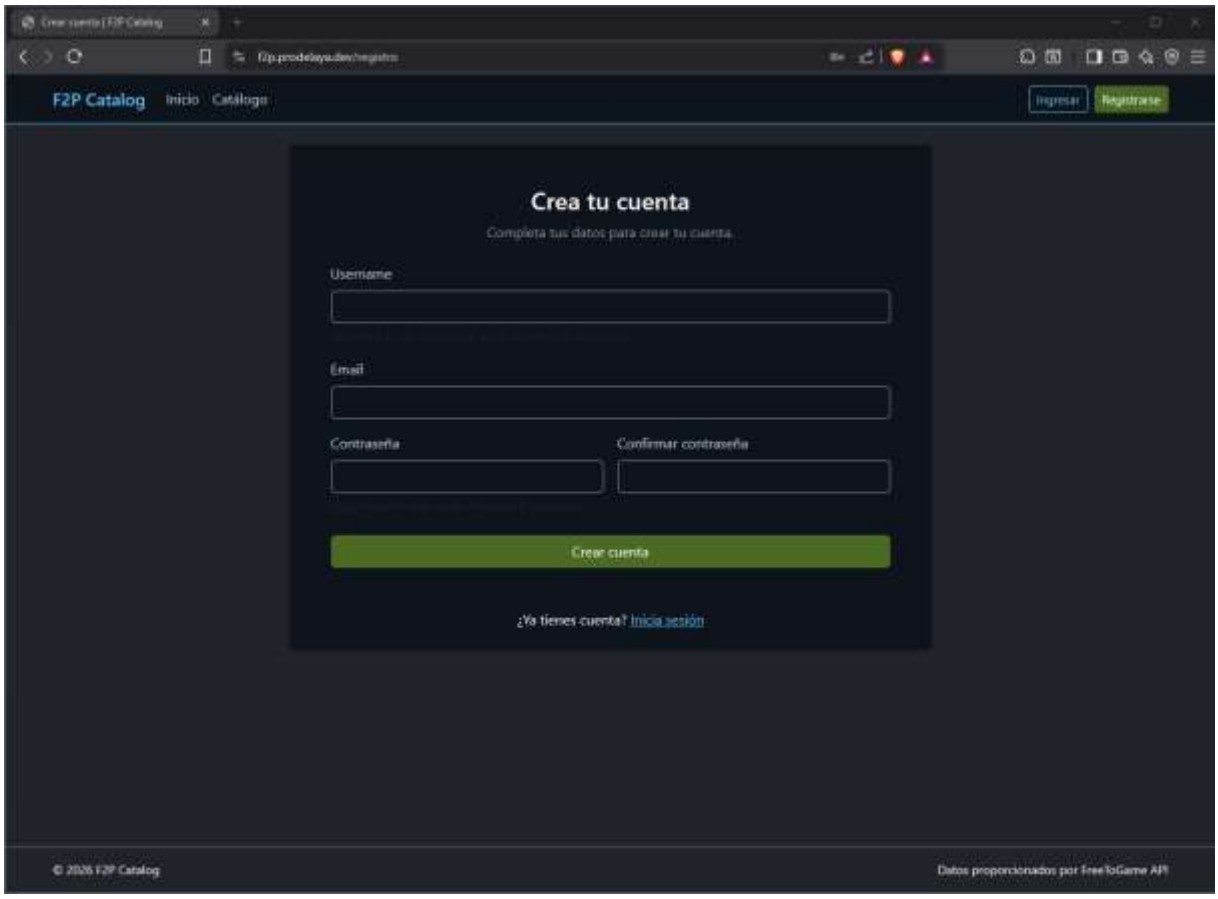


Figura 12. Pantallas de registro con textos revisados. Fuente: elaboración propia.

En local, la aplicación se abre en http://localhost:5000 después de levantar Docker Compose. En producción se documenta el acceso mediante la URL pública del despliegue. Al entrar, el usuario ve la página de inicio con un resumen del proyecto y juegos destacados.

10.2 Consultar catálogo

9. Acceder al enlace Catálogo de la barra superior.
10. Usar el campo de búsqueda si se conoce parte del nombre o descripción.
11. Seleccionar género, plataforma o publisher si se quiere reducir el listado.
12. Cambiar el orden si interesa ver juegos por valoración, reseñas o fecha.
13. Usar la paginación inferior para moverse entre resultados.
14. Pulsar en una tarjeta para abrir la ficha del juego.

10.3 Ver ficha de juego

La ficha muestra portada, etiquetas de género/plataforma, descripción, desarrollador, publisher, fecha de lanzamiento, requisitos mínimos, capturas y reseñas. Si hay enlaces externos disponibles, se puede abrir la web del juego o la ficha en FreeToGame.

10.4 Registro e inicio de sesión

15. Pulsar Registrarse en la barra de navegación.

16. Introducir username, email, contraseña y confirmación.
17. Corregir los errores si el sistema indica datos inválidos o duplicados.
18. Tras el registro, iniciar sesión con email y contraseña.
19. Comprobar que en la barra aparece el nombre de usuario, Mi biblioteca y Cerrar sesión.

10.5 Gestionar biblioteca personal

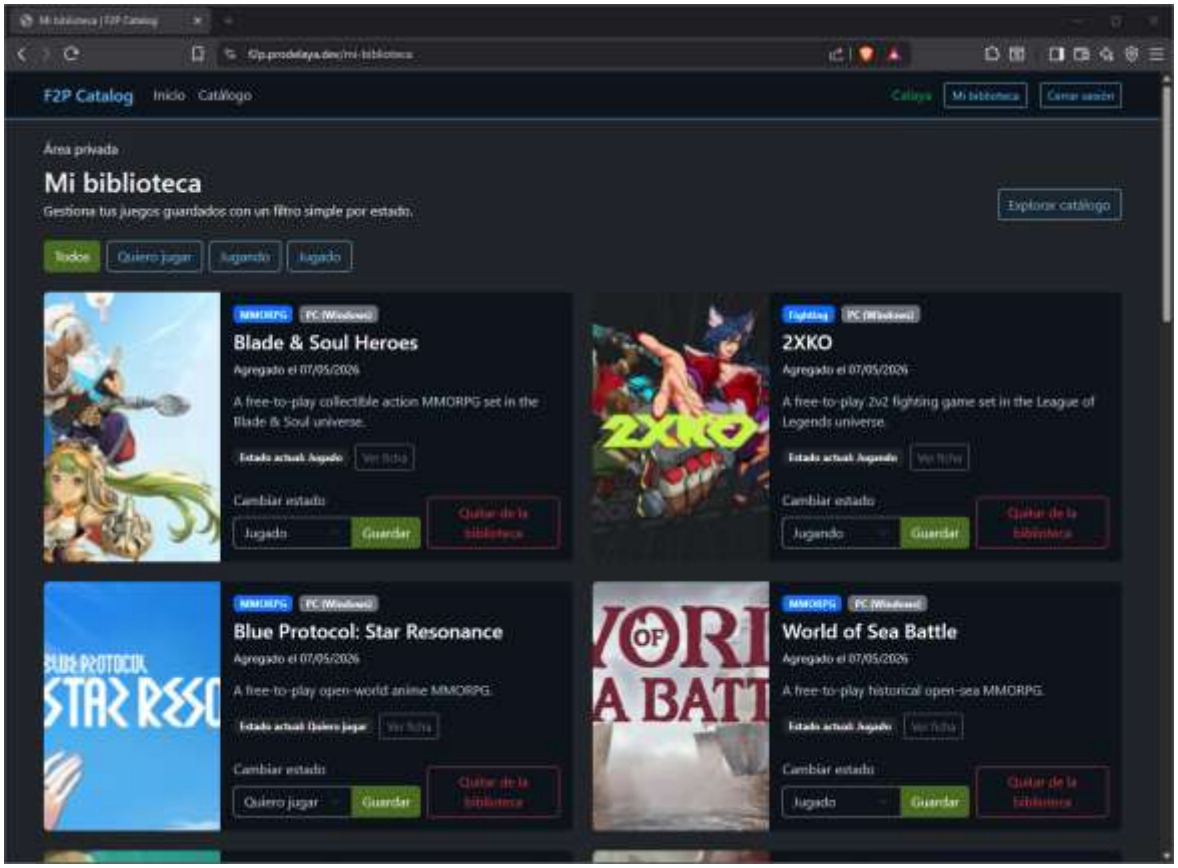


Figura 13. Vista de biblioteca personal con estados de juego usando un usuario de demostración. Fuente: elaboración propia.

20. Iniciar sesión.
21. Abrir la ficha de un juego.
22. Pulsar Agregar a mi biblioteca.
23. Cambiar el estado desde la ficha o desde Mi biblioteca.
24. Usar los filtros de biblioteca para ver todos, quiero jugar, jugando o jugado.
25. Quitar el juego si ya no interesa conservarlo.

10.6 Publicar y editar reseñas

26. Iniciar sesión y abrir la ficha del juego.
27. Seleccionar valoración entre 1 y 5.
28. Escribir una reseña de entre 10 y 1000 caracteres.
29. Publicar la reseña.
30. Editarla si se quiere cambiar texto o valoración.
31. Eliminarla si se desea retirarla.

Solo se permite una reseña por usuario y juego. Esta regla evita que un mismo usuario altere demasiado la valoración de un juego escribiendo varias reseñas.

10.7 Perfil de usuario

El perfil muestra datos básicos de la cuenta, número de juegos en biblioteca, número de reseñas y últimas reseñas escritas. Desde ahí se puede volver al catálogo o a la biblioteca personal.

10.8 Panel de administración

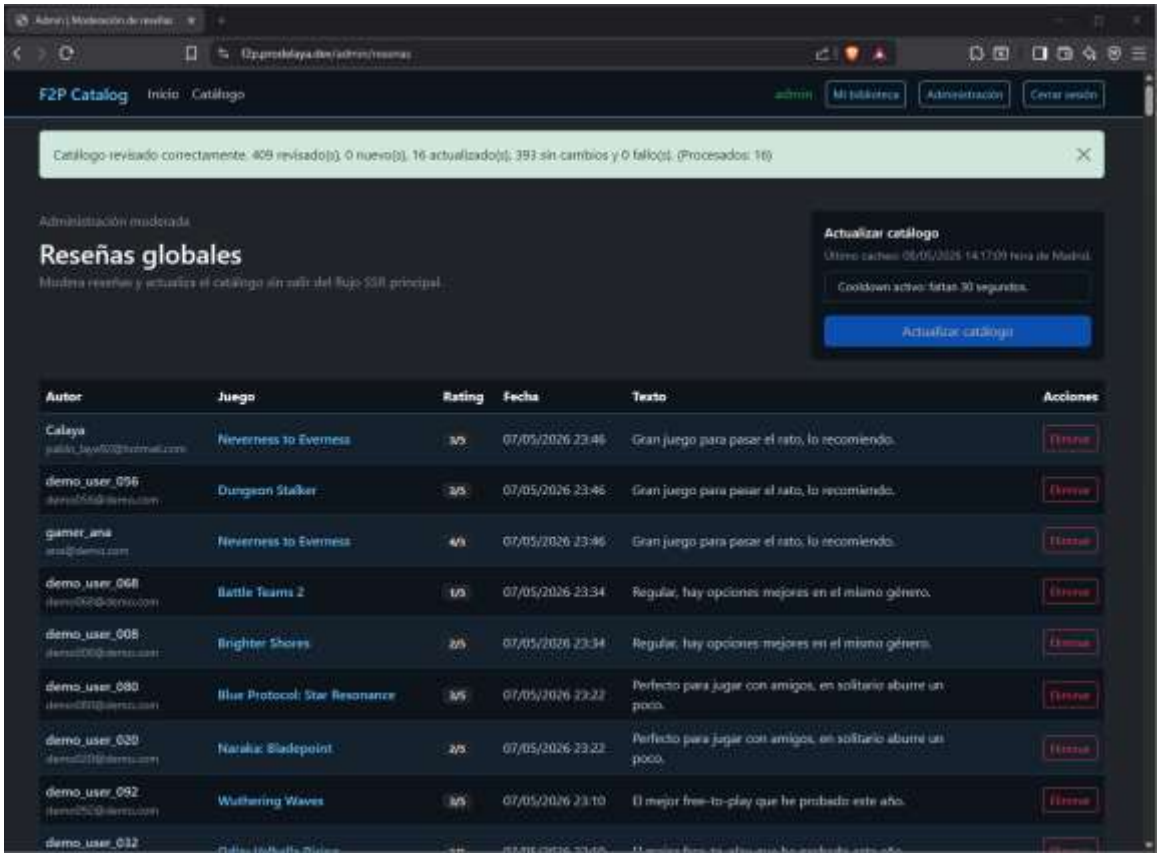


Figura 14. Panel de administración con actualización de catálogo y moderación de reseñas. Fuente: elaboración propia.

El panel de administración solo aparece para usuarios con rol admin. Permite ver todas las reseñas, eliminar reseñas y actualizar el catálogo desde FreeToGame. La actualización tiene un cooldown de 30 segundos para evitar pulsaciones repetidas.

Problema frecuente	Causa probable	Solución
No puedo escribir reseña	No se ha iniciado sesión o ya existe una reseña propia.	Iniciar sesión o editar la reseña existente.
No aparece Administración	La cuenta no tiene is_admin=True.	Usar la cuenta admin creada por seed.
El formulario da error de sesión	Token CSRF caducado o página antigua.	Recargar página y volver a enviar.

Problema frecuente	Causa probable	Solución
No hay resultados en catálogo	Filtros demasiado restrictivos.	Limpiar filtros y buscar de nuevo.

10.9 Recorrido recomendado para una demo

Para enseñar la aplicación en pocos minutos conviene seguir un orden que muestre primero lo público y después lo privado. Así se entiende mejor la diferencia entre visitante, usuario registrado y administrador. Este recorrido también sirve como guion de pruebas manuales rápidas antes de presentar.

32. Abrir la home y explicar que los datos vienen del catálogo cacheado.
33. Entrar al catálogo, buscar un juego y aplicar un filtro de género o plataforma.
34. Abrir la ficha de un juego y enseñar descripción, requisitos, capturas y reseñas.
35. Iniciar sesión con un usuario normal y añadir el juego a la biblioteca.
36. Cambiar el estado del juego a jugando o jugado.
37. Crear una reseña válida y después editarla.
38. Entrar al perfil para enseñar contadores y últimas reseñas.
39. Cerrar sesión y entrar como administrador.
40. Abrir el panel admin, mostrar reseñas y explicar el botón de actualización de catálogo.

Este recorrido evita saltar de una pantalla a otra sin contexto. Además, permite enseñar que la interfaz no es solo visual: cada botón está conectado a rutas Flask, validaciones, modelos y permisos.

11. Manual técnico

11.1 Requisitos previos

- Git para clonar el repositorio.
- Docker y Docker Compose para ejecutar web y base de datos de forma sencilla.
- Opcionalmente Python 3.11 si se quiere ejecutar fuera de Docker.
- Conexión a internet para la primera carga de juegos desde FreeToGame.
- Editor de código, por ejemplo VS Code.

11.2 Instalación local con Docker

La versión entregada debe quedar asociada a una rama, etiqueta o commit del repositorio. Para la entrega final se recomienda etiquetar el estado como entrega-final-mayo-2026 o anotar el hash de commit en el README.

```
git clone git@github.com:Prodelaya/Proyecto-DAW-Juegos-F2P.git
cd Proyecto-DAW-Juegos-F2P
cp .env.example .env
docker compose up --build
```

Cuando los contenedores estén activos, la aplicación se abre en http://localhost:5000. La base de datos PostgreSQL queda en el contenedor db y usa el volumen pgdata.

11.3 Carga de datos inicial

```
docker compose exec web python seeds/seed_all.py
```

Este comando ejecuta los seeds en orden: juegos, usuarios, reseñas y biblioteca. La primera carga puede tardar porque se consulta el detalle de muchos juegos. Si se ejecuta otra vez, el sistema intenta no duplicar datos y devuelve contadores de procesados y fallidos.

La primera carga puede tardar varios minutos porque consulta detalles de muchos juegos. Si FreeToGame no responde, se recomienda revisar conexión, logs y reintentar más tarde. Los usuarios demo se crean desde seeds/seed_users.py usando valores de ejemplo; las contraseñas reales no deben escribirse en la memoria ni en capturas.

11.4 Ejecución fuera de Docker

```
python -m venv .venv
source .venv/bin/activate  # en Windows: .venv\Scripts\activate
pip install -r requirements.txt
python seeds/seed_all.py
flask run
```

Fuera de Docker hay que revisar DATABASE_URL. Si no se define, config.py usa una base SQLite local en instance/f2p_catalog.db. Esta opción puede servir para pruebas rápidas, aunque el proyecto se plantea principalmente con PostgreSQL.

11.5 Archivos de configuración

Variable	Descripción
DATABASE_URL	Cadena de conexión a PostgreSQL o SQLite.
SECRET_KEY	Clave secreta de Flask para sesiones y CSRF.
FLASK_ENV	development o production.
FREETOGAME_API_URL	URL base de la API externa.
ADMIN_EMAIL / ADMIN_PASSWORD	Credenciales del admin demo desde .env; no se publican valores reales.
POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB	Credenciales del contenedor PostgreSQL.
Puerto local	La web se prueba en http://localhost:5000.

11.6 Mantenimiento técnico

Tarea	Comando / acción
Ver contenedores locales	docker compose ps
Ver logs locales	docker compose logs -f web
Recrear contenedores	docker compose up -d --build
Parar entorno	docker compose down
Borrar volumen de BD local	docker compose down -v
Probar conexión API	Revisar logs del seed y conectividad a https://www.freetogame.com/api
Ejecutar seed en producción	docker compose -f docker-compose.prod.yml exec web python seeds/seed_all.py

11.7 Solución de problemas

Síntoma	Posible causa	Corrección
La web no arranca	PostgreSQL todavía no acepta conexiones.	Revisar wait_for_postgres, docker compose ps y logs de db/web.
No se cargan juegos	FreeToGame no responde o no hay internet.	Reintentar seed y revisar docker compose logs -f web.
Login admin no funciona	ADMIN_EMAIL/PASSWORD no coinciden con seed.	Actualizar .env y reejecutar seed_users/seed_all.
Error CSRF	Formulario antiguo o token ausente.	Recargar página y verificar csrf_input en template.
Catálogo lento al actualizar	Primera carga o muchos cambios.	Esperar; la carga inicial puede

Síntoma	Posible causa	Corrección
		tardar varios minutos.
Error con varios workers	Condición de carrera detectada.	Usar 1 worker y 4 threads como en docker-compose.prod.yml.
DATABASE_URL incorrecta	Variable mal escrita o contenedor db no accesible.	Comprobar .env y nombre de servicio db.

11.8 Procedimiento seguro de actualización

Cuando se actualice el código del servidor, conviene seguir un procedimiento sencillo para evitar dejar la web en un estado raro. Primero se revisan cambios y variables, luego se reconstruyen contenedores, después se comprueban logs y por último se ejecutan los seeds si realmente hace falta. No se debe ejecutar seed_all sin saber si se necesita, porque la primera carga puede tardar y consume la API externa.

Paso	Acción	Comprobación
1	Actualizar código del repositorio en el servidor.	La rama o commit corresponde a la versión que se quiere desplegar.
2	Revisar .env de producción.	No contiene valores por defecto ni secretos vacíos.
3	Reconstruir con docker compose de producción.	Los contenedores quedan levantados.
4	Leer logs de web y db.	No hay errores repetidos al arrancar.
5	Probar URL pública.	Home, catálogo y login funcionan.
6	Ejecutar seeds si procede.	El resumen final no muestra fallos importantes.

11.9 Estructura de plantillas y parciales

Las plantillas están divididas para no repetir código. base.html define el esqueleto general, los partials agrupan elementos comunes y cada carpeta contiene las pantallas de una funcionalidad. Esta separación permite que un cambio en la navbar, por ejemplo, afecte a toda la web sin tocar cada pantalla una a una.

Carpeta / archivo	Función
templates/base.html	Layout HTML principal, carga Bootstrap, CSS y JS.
partials/navbar.html	Navegación con enlaces según sesión y rol.
partials/game_card.html	Tarjeta reutilizable de juego.
partials/pagination.html	Controles de paginación conservando filtros.

Carpeta / archivo	Función
partials/csrf_input.html	Input hidden con token CSRF para formularios POST.
games/detail.html	Ficha completa con datos, biblioteca y reseñas.
admin/reviews.html	Panel de moderación y actualización de catálogo.

12. Conclusiones y líneas futuras

12.1 Conclusiones

El proyecto alcanza el objetivo principal de crear una aplicación web funcional para descubrir, valorar y organizar juegos free-to-play. La parte más difícil no fue crear una pantalla concreta, sino mantener conectadas rutas, formularios, plantillas y base de datos sin romper los flujos de usuario. Esa fue la principal enseñanza técnica del proyecto.

Una de las partes más interesantes fue integrar una API externa sin depender de ella en cada petición. Cachear los juegos en PostgreSQL hizo que el catálogo pudiera filtrarse y paginarse desde datos propios. También ayudó a entender mejor la diferencia entre consumir una API para poblar datos y exponer una API propia al frontend.

Otra dificultad fue mantener el frontend sin romper el backend. Al trabajar con Jinja2, cada variable, name de input, url_for o token CSRF forma parte de un contrato. Cambiar solo el aspecto visual puede romper una ruta si se elimina un hidden input o se cambia un nombre de campo. Esta parte fue bastante práctica para entender cómo se conectan HTML y Flask.

El despliegue también aportó aprendizaje. Se vio que una configuración que funciona en local puede necesitar ajustes en producción, como ocurrió con los workers de Gunicorn. La decisión de usar Docker Compose, Gunicorn, PostgreSQL y Cloudflare Tunnel resultó más adecuada que intentar usar Vercel, porque el proyecto no es un frontend estático sino una aplicación Flask SSR.

12.2 Líneas futuras

- Añadir migraciones con Flask-Migrate/Alembic para evolucionar el esquema sin depender de db.create_all().
- Crear pruebas automatizadas unitarias y de integración para rutas, modelos y seeds.
- Mejorar el sistema de búsqueda con filtros combinados más avanzados y quizá orden por popularidad externa si la fuente lo permite.
- Añadir recuperación de contraseña y verificación de email.
- Permitir listas personalizadas más allá de los tres estados actuales.
- Crear un sistema de reportes de reseñas para que los usuarios ayuden a moderar.
- Añadir estadísticas de catálogo y uso para administración.
- Programar actualizaciones periódicas del catálogo con una tarea controlada, no solo manual.
- Mejorar accesibilidad con revisión de contraste, navegación con teclado y textos alternativos más completos.
- Preparar copias de seguridad automáticas de PostgreSQL en producción.

En general, el proyecto ha servido para practicar muchas piezas de una aplicación web real sin perder el control del alcance: rutas Flask, plantillas Jinja2, validaciones, sesiones, PostgreSQL, Docker y despliegue. La memoria y la defensa deben centrarse en mostrar esas piezas funcionando con ejemplos concretos, no en presentar la aplicación como si fuera un producto comercial terminado.

12.3 Relación con módulos de DAW

El proyecto toca varios contenidos del ciclo. En desarrollo de servidor se trabaja con rutas, formularios, sesiones y acceso a base de datos. En diseño de interfaces se aplican plantillas, responsive y usabilidad. En despliegue se usan contenedores, variables de entorno y configuración de servidor. En bases de datos se diseñan tablas, claves, restricciones y relaciones.

Módulo / área	Evidencia en el proyecto
Desarrollo web en entorno servidor	create_app(), blueprints, Flask-Login, validaciones y renderizado Jinja2.
Desarrollo web en entorno cliente	Templates, Bootstrap, CSS propio, navegación y JavaScript puntual.
Diseño de interfaces web	Home, catálogo, ficha, biblioteca, perfil, admin y estados vacíos.
Bases de datos	Modelos User, Game, Review y UserLibrary con relaciones y constraints.
Despliegue de aplicaciones	Docker Compose, Gunicorn, PostgreSQL 15, variables .env y logs.
Proyecto	Planificación por fases, reparto, pruebas manuales, memoria y defensa.

Visto así, F2P Catalog funciona como un proyecto integrador. No destaca por una tecnología muy novedosa, sino por combinar herramientas conocidas de forma ordenada. Esa es una parte importante del aprendizaje: muchas veces lo difícil no es usar una librería, sino conectar todas las piezas sin que unas rompan a otras.

Bibliografía y webgrafía

Newzoo. (2024). Global Games Market Report 2024: Free version. Newzoo. https://newzoo.com/resources/trend-reports/newzoos-global-games-market-report-2024-free-version

FreeToGame. (s. f.). Free-To-Play Games Database API. https://www.freetogame.com/api-doc

Bootstrap Team. (s. f.). Bootstrap Documentation. https://getbootstrap.com/docs/ (Consulta: 8 de mayo de 2026).

Docker, Inc. (s. f.). Docker Documentation. https://docs.docker.com/ (Consulta: 8 de mayo de 2026).

Flask-WTF Authors. (s. f.). Flask-WTF Documentation. https://flask-wtf.readthedocs.io/ (Consulta: 8 de mayo de 2026).

FreeToGame. (s. f.). FreeToGame API Docs. https://www.freetogame.com/api-doc (Consulta: 8 de mayo de 2026).

Gunicorn Developers. (s. f.). Gunicorn Documentation. https://docs.gunicorn.org/en/stable/ (Consulta: 8 de mayo de 2026).

maxcountryman. (s. f.). Flask-Login Documentation. https://flask-login.readthedocs.io/ (Consulta: 8 de mayo de 2026).

OWASP Foundation. (s. f.). OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/ (Consulta: 8 de mayo de 2026).

Pallets. (s. f.). Flask Documentation. https://flask.palletsprojects.com/ (Consulta: 8 de mayo de 2026).

Pallets-Eco. (s. f.). Flask-SQLAlchemy Documentation. https://flask-sqlalchemy.palletsprojects.com/ (Consulta: 8 de mayo de 2026).

PostgreSQL Global Development Group. (s. f.). PostgreSQL 15 Documentation. https://www.postgresql.org/docs/15/ (Consulta: 8 de mayo de 2026).

Python Software Foundation. (s. f.). Python 3.11 Documentation. https://docs.python.org/3.11/ (Consulta: 8 de mayo de 2026).

Unión Europea. (2016). Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, Reglamento General de Protección de Datos. https://eur-lex.europa.eu/eli/reg/2016/679/oj (Consulta: 8 de mayo de 2026).

Anexos

Además de los comandos, los anexos recogen evidencias técnicas breves que no conviene mezclar con la explicación principal: salida de Docker, seed de juegos, ejemplo de variables sin secretos y resumen de entidades.

```
$ docker compose exec web flask seed_all
[games] /api/games: 409 registros recibidos
[details] Raven2: requisitos y 3 screenshots guardados
[sync] creados: 0 · actualizados: 27 · sin cambios: 382 · fallidos: 0
[reviews] datos semilla de demostración cargados para pruebas
[done] seed completado en 03:42 min (entorno de demo)

$ docker compose ps
NAME            SERVICE   STATUS          PORTS
f2p-web-1       web       Up 2 minutes    127.0.0.1:5000->5000/tcp
f2p-db-1        db        Up 2 minutes    5432/tcp
```

Figura 15. Evidencia anexada de seed y contenedores activos para la defensa técnica. Fuente: elaboración propia.

Anexo I. Resumen de comandos principales

```
# Desarrollo local
docker compose up --build
docker compose exec web python seeds/seed_all.py

# Producción
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml logs -f web

docker compose -f docker-compose.prod.yml exec web python seeds/seed_all.py
```

Anexo II. Resumen de entidades

Entidad	Responsabilidad	Relaciones
User	Cuenta, login, rol admin y datos básicos.	1:N con Review y UserLibrary.
Game	Juego cacheado desde FreeToGame.	1:N con Review y UserLibrary.
Review	Valoración y texto de usuario sobre un juego.	N:1 con User y Game.
UserLibrary	Estado privado de un juego para un usuario.	N:1 con User y Game.

Anexo III. Glosario breve

Término	Explicación en el proyecto
SSR	Renderizado en servidor: Flask devuelve HTML ya generado.
Blueprint	Agrupación de rutas Flask por funcionalidad.
Seed	Script que crea o actualiza datos iniciales en la base de datos.
CSRF	Ataque que intenta enviar formularios en nombre del usuario; se mitiga con tokens.
ORM	Capa que permite usar clases Python para trabajar con tablas.
versión inicial	Producto mínimo viable: versión reducida pero funcional.
Cooldown	Tiempo de espera antes de repetir una acción, como actualizar catálogo.
Cache	Copia local de datos externos para no consultarlos siempre.